



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Multimodal Embedding

INFS 4205/7205 Advanced Techniques for High Dimensional Data

Yadan Luo

School of EECS, The University of Queensland

Atlassian on Campus: Internship to Graduate Pathways

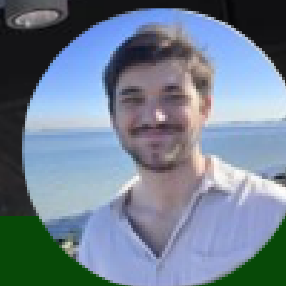
Share

Post

Insights from Industry

Thu 12 Mar 2026, 10:30 am - 11:30 am

Hawken Engineering Building (50), St Lucia



Djamahl Etchegaray

⚠ There are no places remaining for this event

+ Join the waiting list



Session

⌚ Thu 12 Mar 2026, 10:30 am - 11:30 am

⚠ No places remaining

Other information

Email:

employability.events@eait.uq.edu.au

More 'Insights from Industry' events

Engineering Careers in the Water Sector

Outline

After today's class, you should be able to...

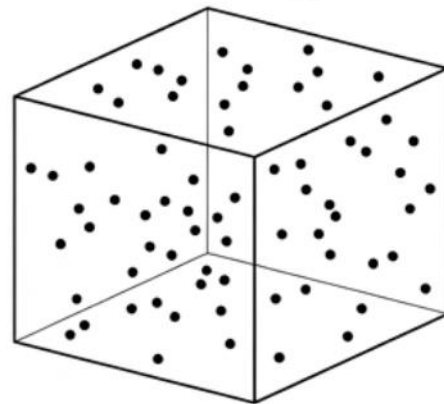
- Describe why the Transformer became a **universal encoder** across text, vision, and other modalities
- Explain the **building blocks** of the Transformer architecture to a non-technical person;
- Describe how **positional encoding** tells the model where each token is; and how **self-attention** works intuitively and mathematically, including queries, keys, values, and scaled dot-product attention

Week 1 Recap... (The Blessing of Dimensionality)

- High-dim space is extremely **Sparse**!

Euclidean distance **fail** in raw data space

The Manifold Hypothesis

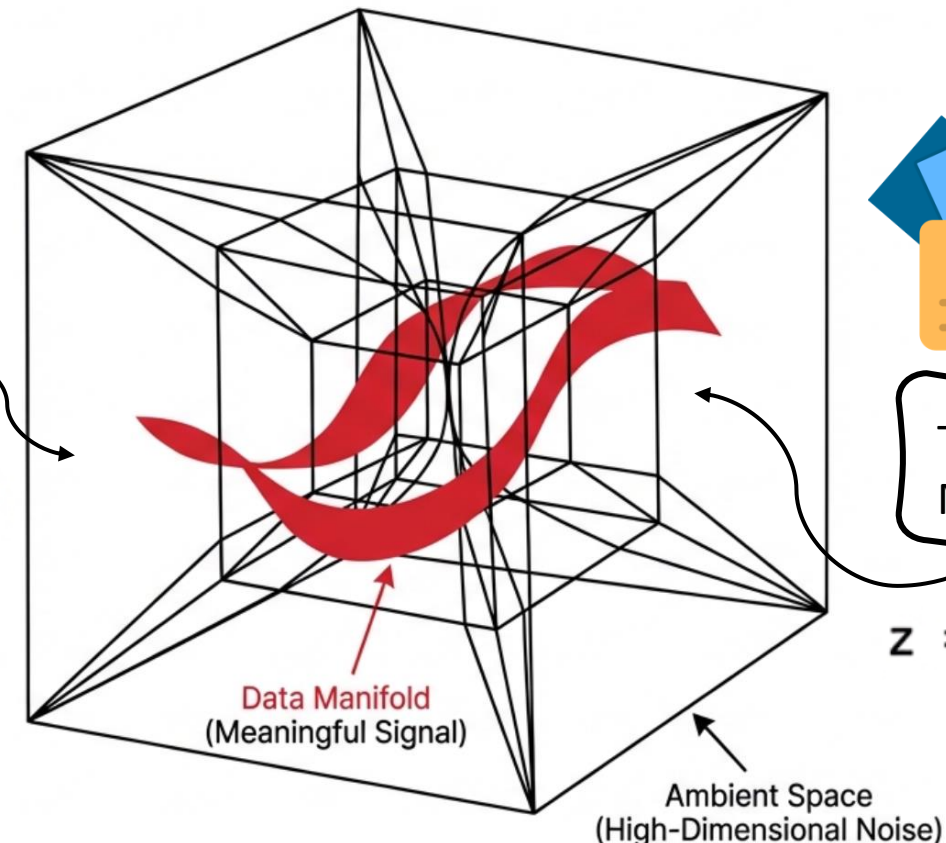


Raw High-Dimensional Space (Sparse)



Vision Model

$$z = f_{\theta}(x_{\text{raw}})$$



Text Model

$$z = f_{\theta}(x_{\text{raw}})$$

We don't need to model the entire empty space -
> learn the *mapping* to the **manifold**

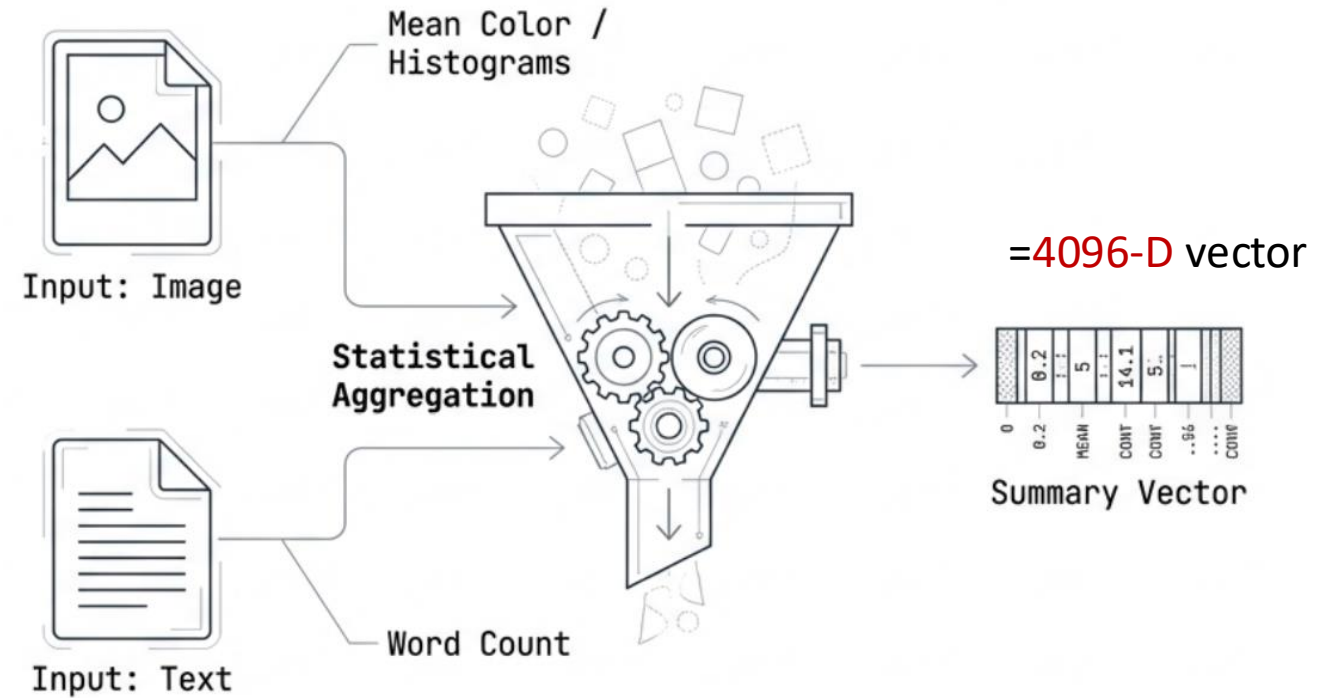
Goal: learn a function $f(x)$ that maps raw input to manifold

Early Feature Extractors: Statistics First

- Before deep learning, we treated data as **bags of statistics**

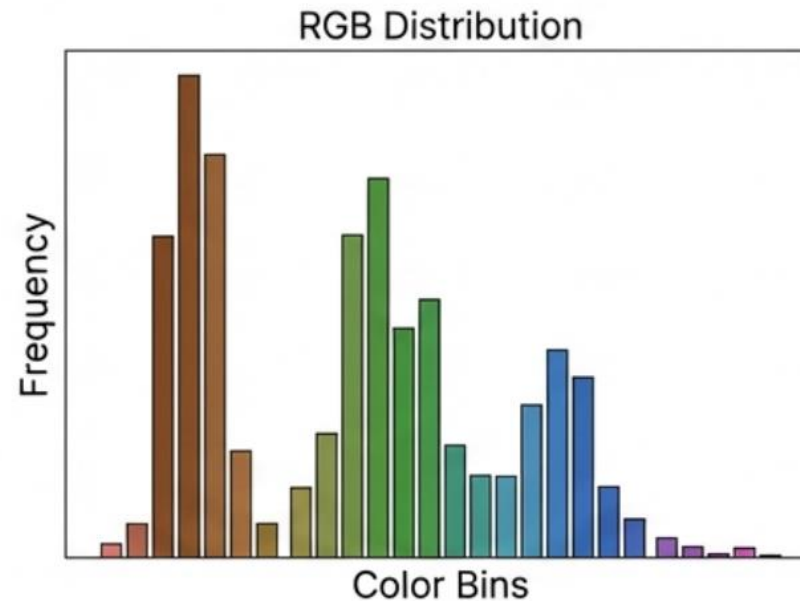
- Summarized data via **global** statistics (distribution, averaging, counting)
- **Aggressive** dimensionality reduction
- Simple, fast, mathematically **interpretable**
- **Failure**: completely ignored spatial structure and sequential context

$(220 \times 160 \times 3) \approx 105,600$ **raw** dimensions per image



$$\phi(\mathbf{x}) = \text{aggregate}(\text{features}(\mathbf{x}))$$

Vision: Color Histograms

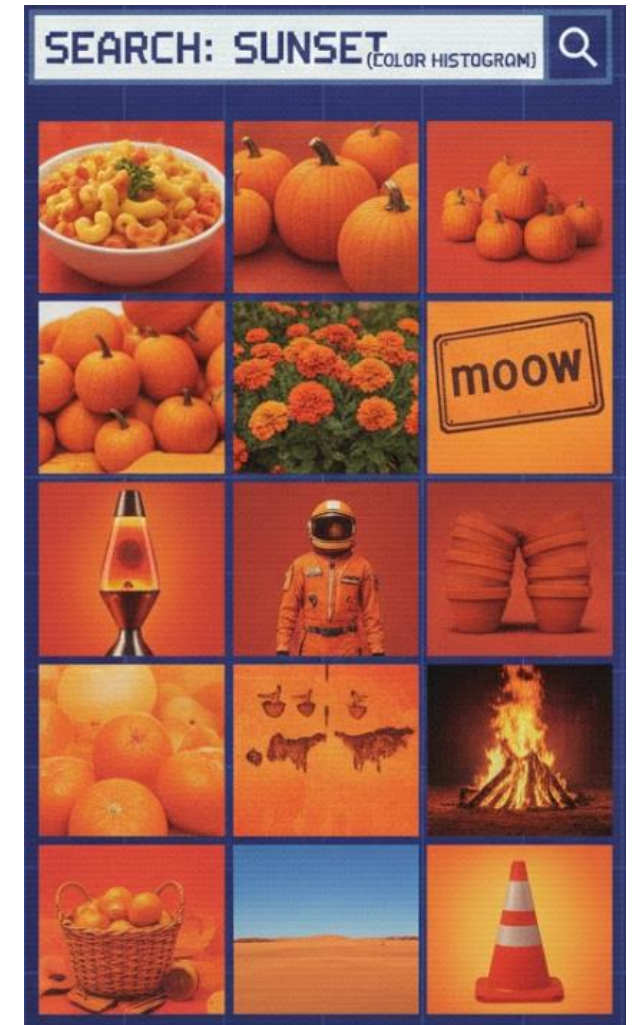


+ invariant to scaling & rotation

But...

$$h_c = \sum_c (\text{indicator}[x_{ij} \in \text{bin}_c])$$

"This worked for early image retrieval—if you searched for “sunset,” you got images with lots of orange. But the model had **no concept of objects, shapes, or layout**. It was “blind” to geometry."



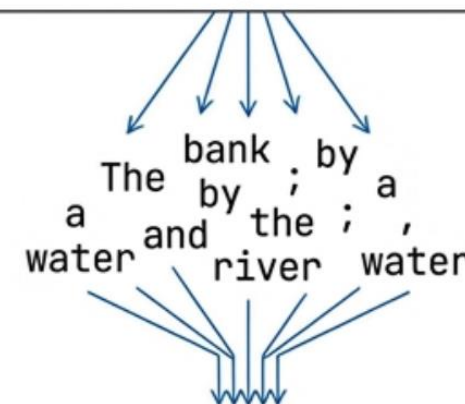
Text: Bag of Words (BoW)

The linguistic equivalent of color histograms.

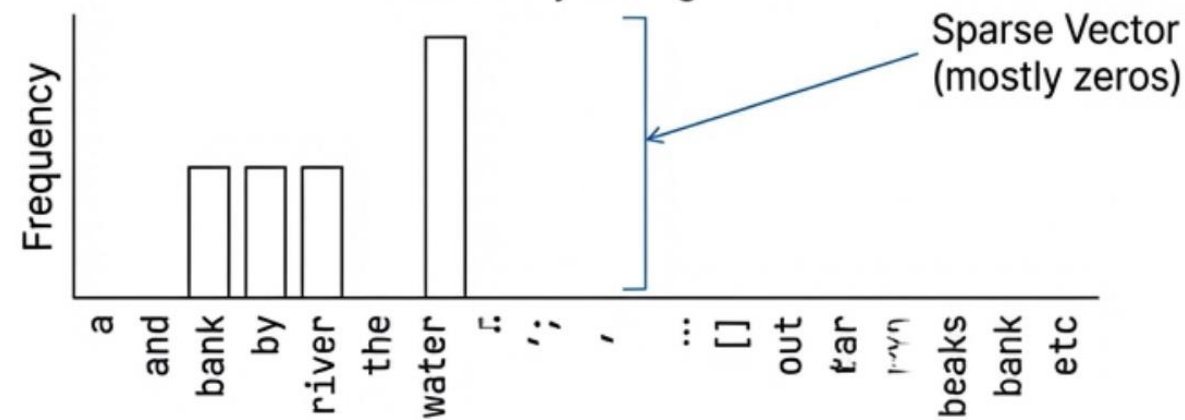
- Counts word occurrences regardless of position.
- “Dog bites man” and “Man bites dog” result in *identical* vectors.
- Produces extremely sparse, high-dimensional vectors.
- Meaning collapses without *syntax or context*.



Input Text: The bank by the river



Vocabulary Histogram

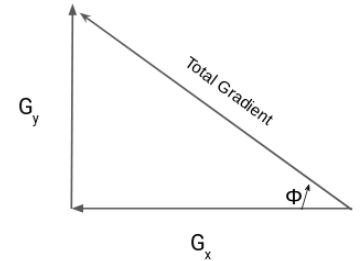
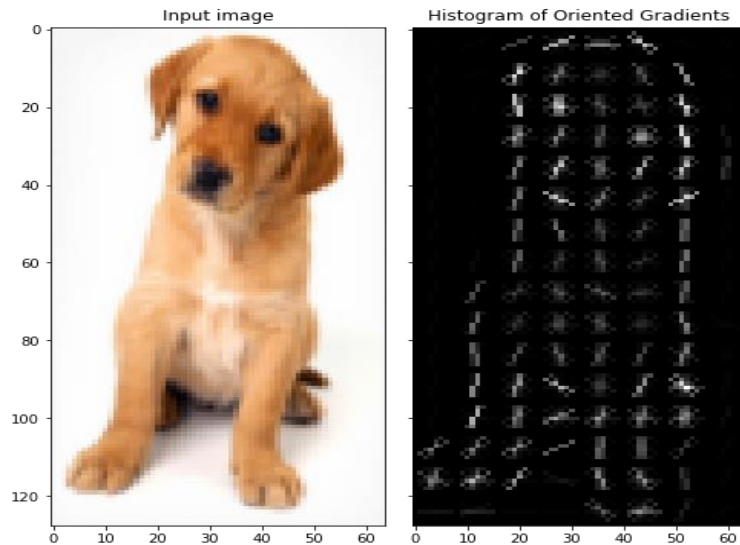
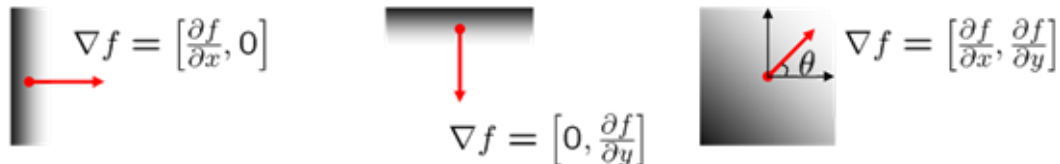


$$v[w] = \text{count}(w, \text{document})$$

Adding Local Structure: HOG and SIFT

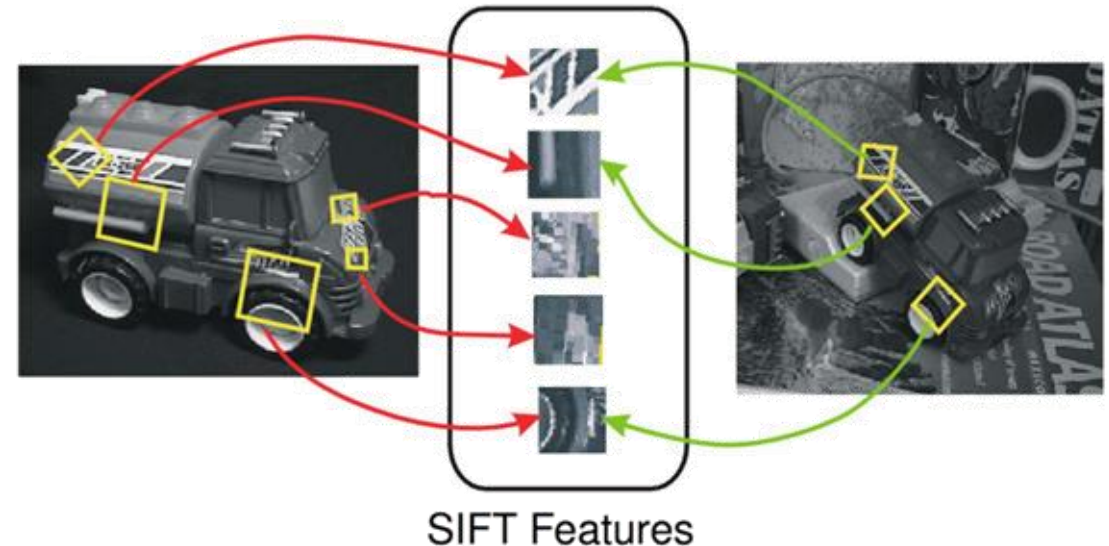
Engineered features to capture local shape

- Captures **local gradient orientations** rather than raw colors.
- Encodes contours, edges, and primitive shapes.
- Relies on a strong **locality assumption** (pixels nearby matter).
- still **hand-designed**



$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

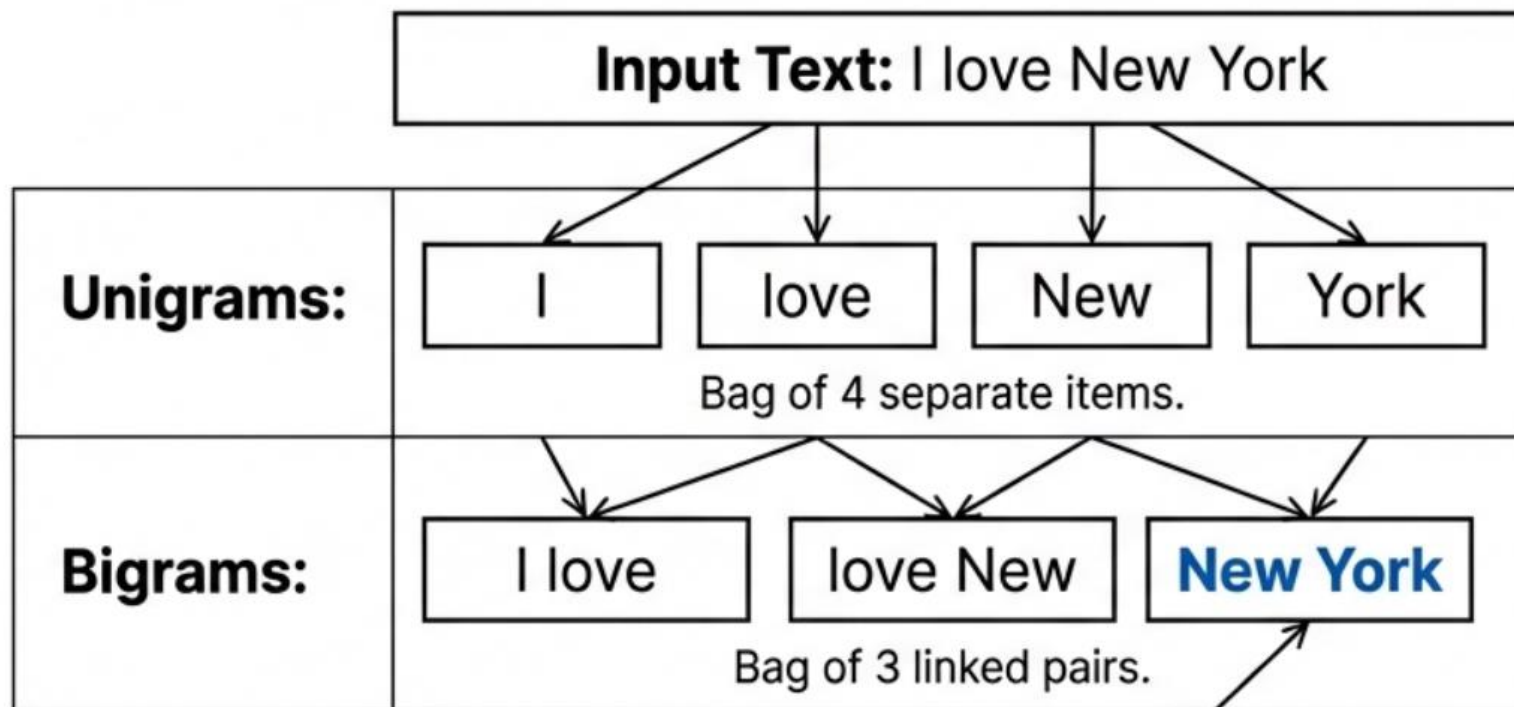
$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$



Text Analogue: n-grams

Attempting to capture short-range context.

- Captures **short-range word order** (e.g., 'not good' vs 'good').
- Slight improvement over Bag-of-Words for **local context**.
- Feature dimension grows exponentially with sequence length (V^n).
- Still produces sparse, brittle representations.



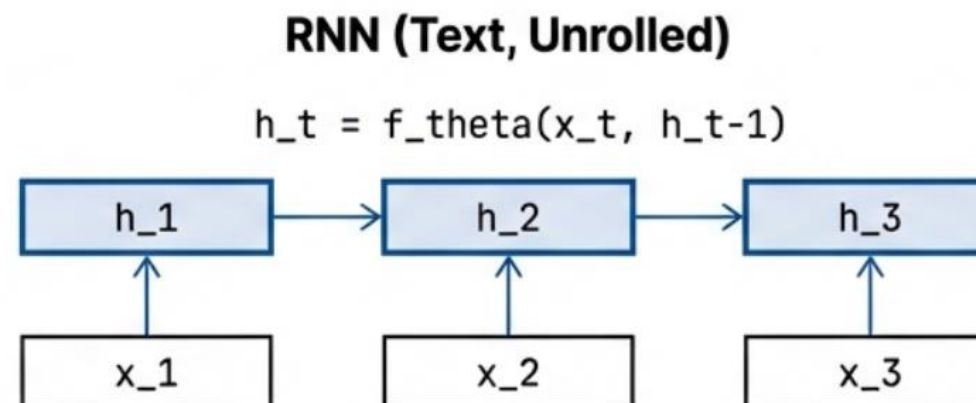
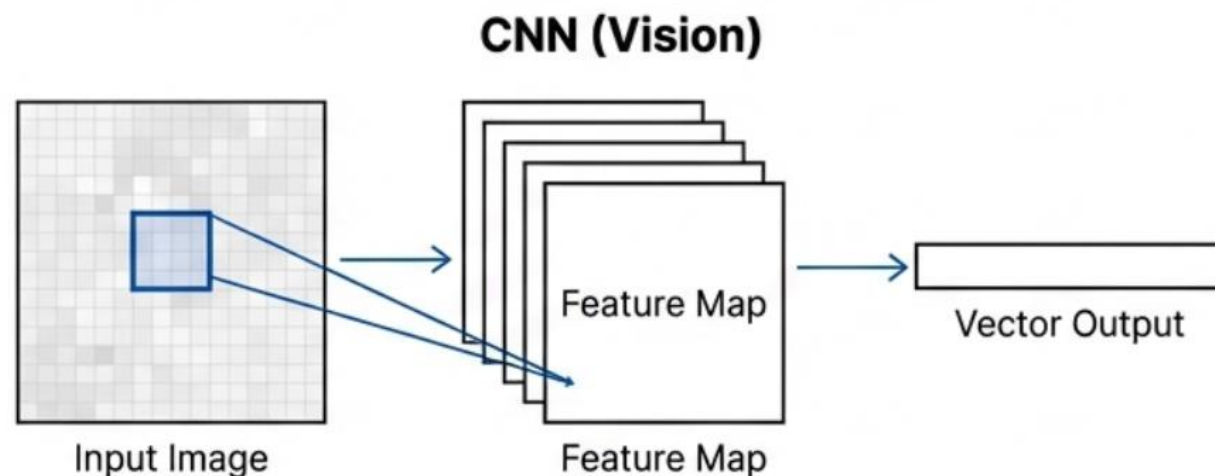
Annotation: "New York" is highlighted to show that the concept is preserved.

$$v[(w_i, w_{i+1})] += 1$$

Learnable Feature Extractors

The Deep Learning Revolution (circa 2012).

- **CNNs:** Convolutional filters learn local spatial patterns (edges $\rightarrow$ textures).
- **RNNs:** Recurrent states capture sequential dependencies over time.
- Architecture encodes strong assumptions (inductive biases).
- Features are optimized via backpropagation.

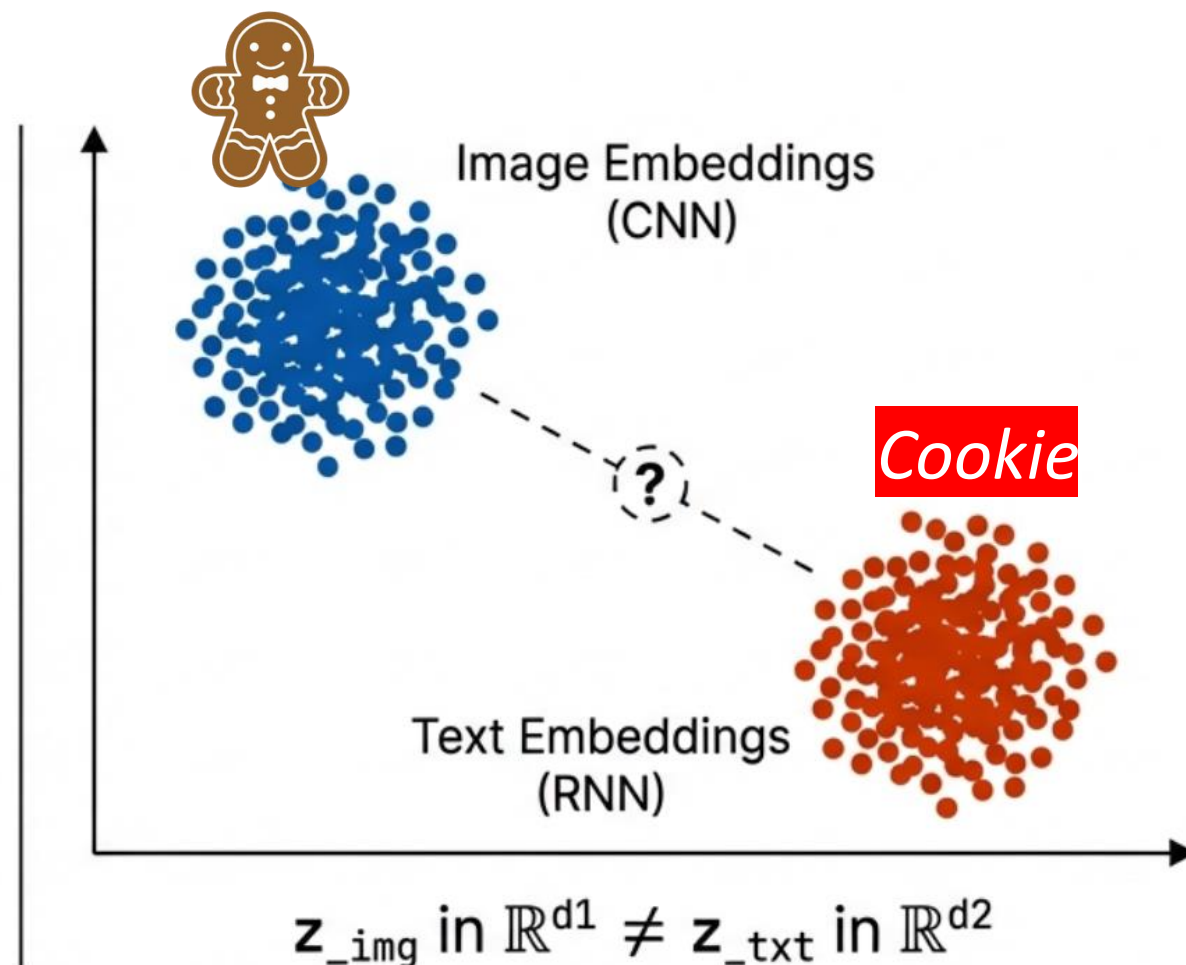


The Universal Feature Encoder: Transformer

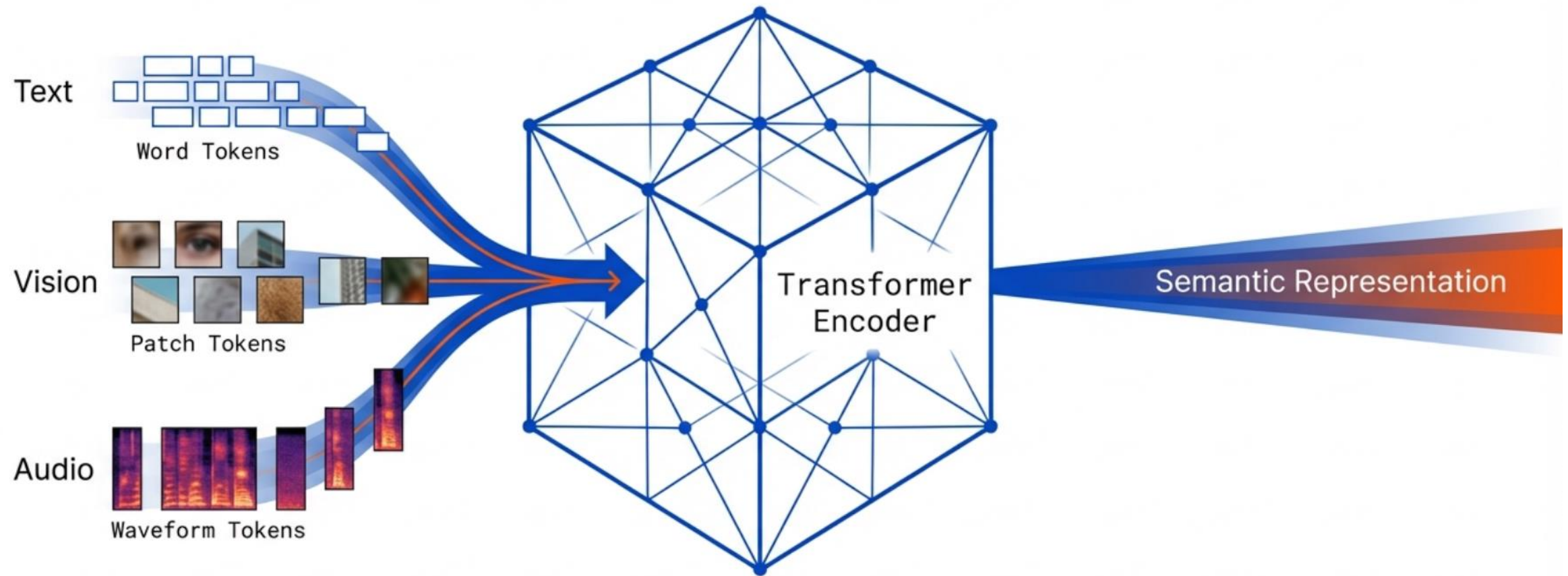
They Spoke Different Language!

Incompatible mathematical spaces.

- Vision and Text use fundamentally different mathematical operations.
- Image embeddings (z_{img}) and text embeddings (z_{txt}) live in disjoint spaces.
- Impossible to perform direct arithmetic (e.g., vector similarity).
- Multimodal interaction requires complex, ad-hoc glue layers.



The Limitation: Modality Silos



We have moved from specialized silos to a unified framework.
Whether pixels, words, or sounds—everything is a **Token**.

What is a Transformer???

(2017)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

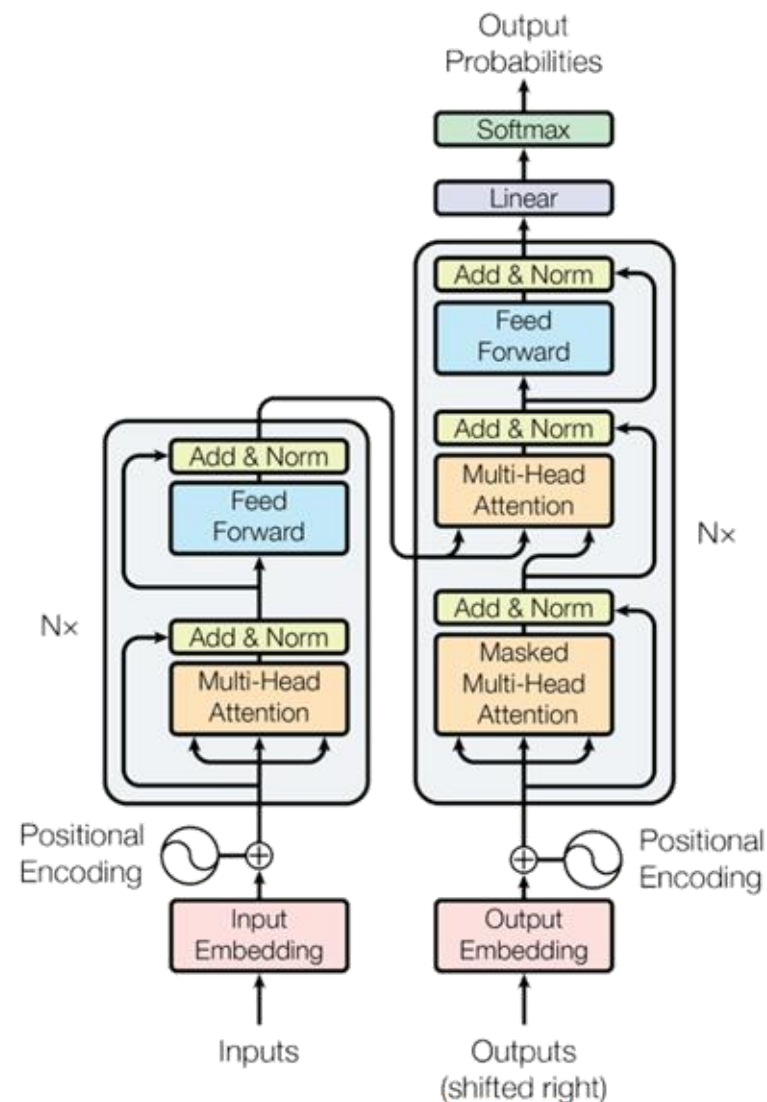


Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

What is a Transformer???

OpenAI

GPT-#, Chapt-GPT

Anthropic

Claude, Sonnet

Google

Gemini

Meta

Llama-#

...

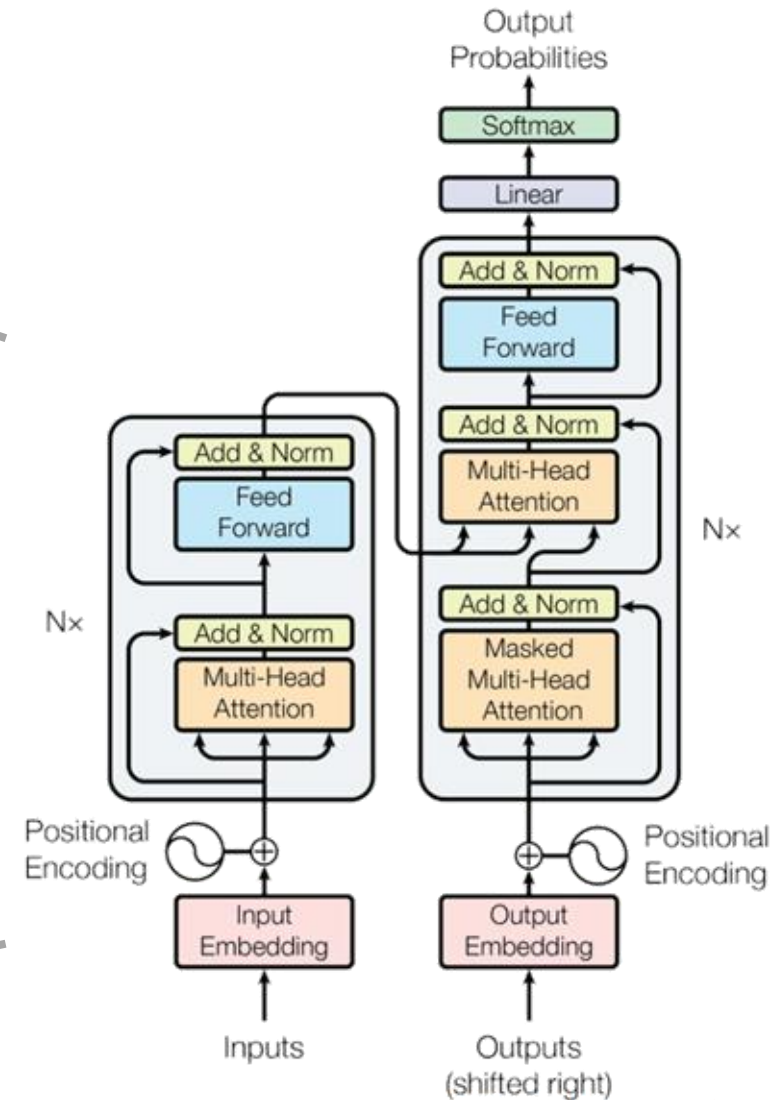


Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

What is a Transformer???

From **high-level** speaking, it follows

- Encoder-Decoder architecture

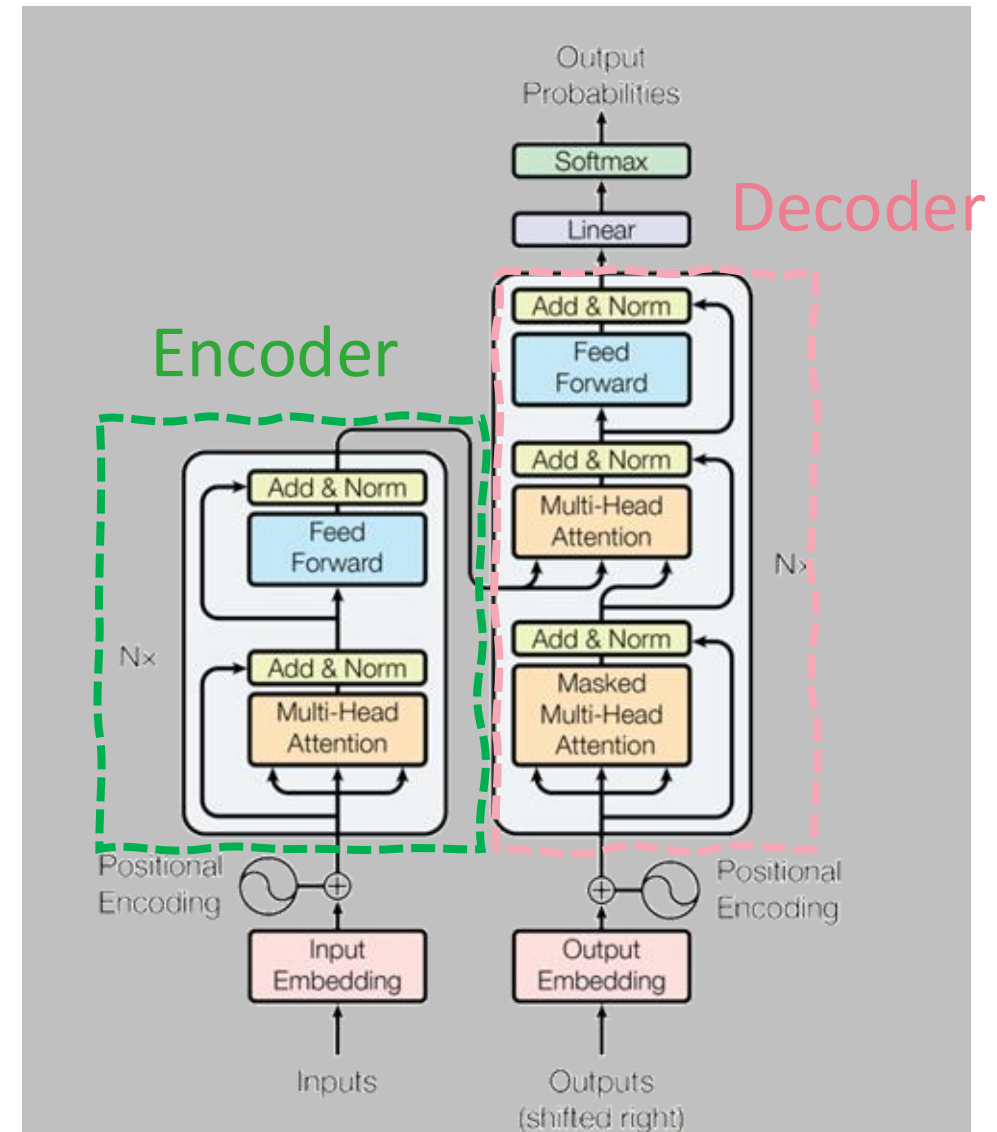
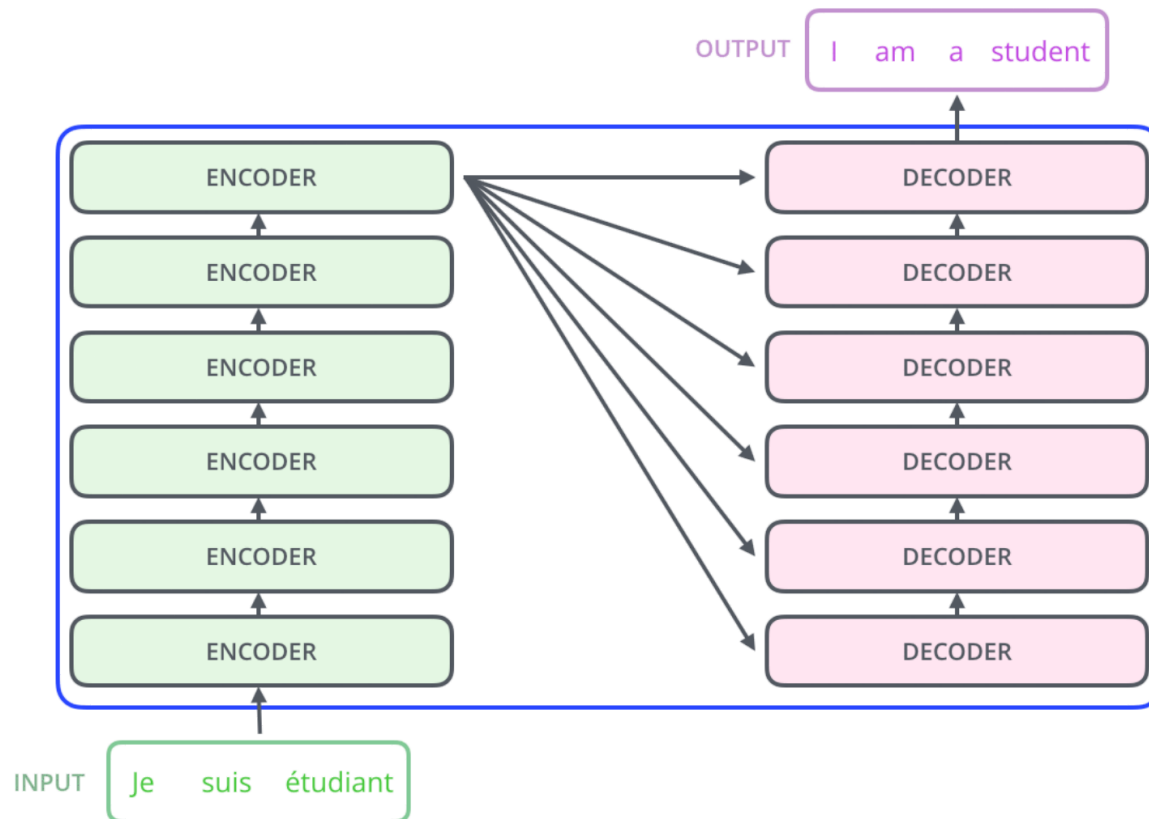
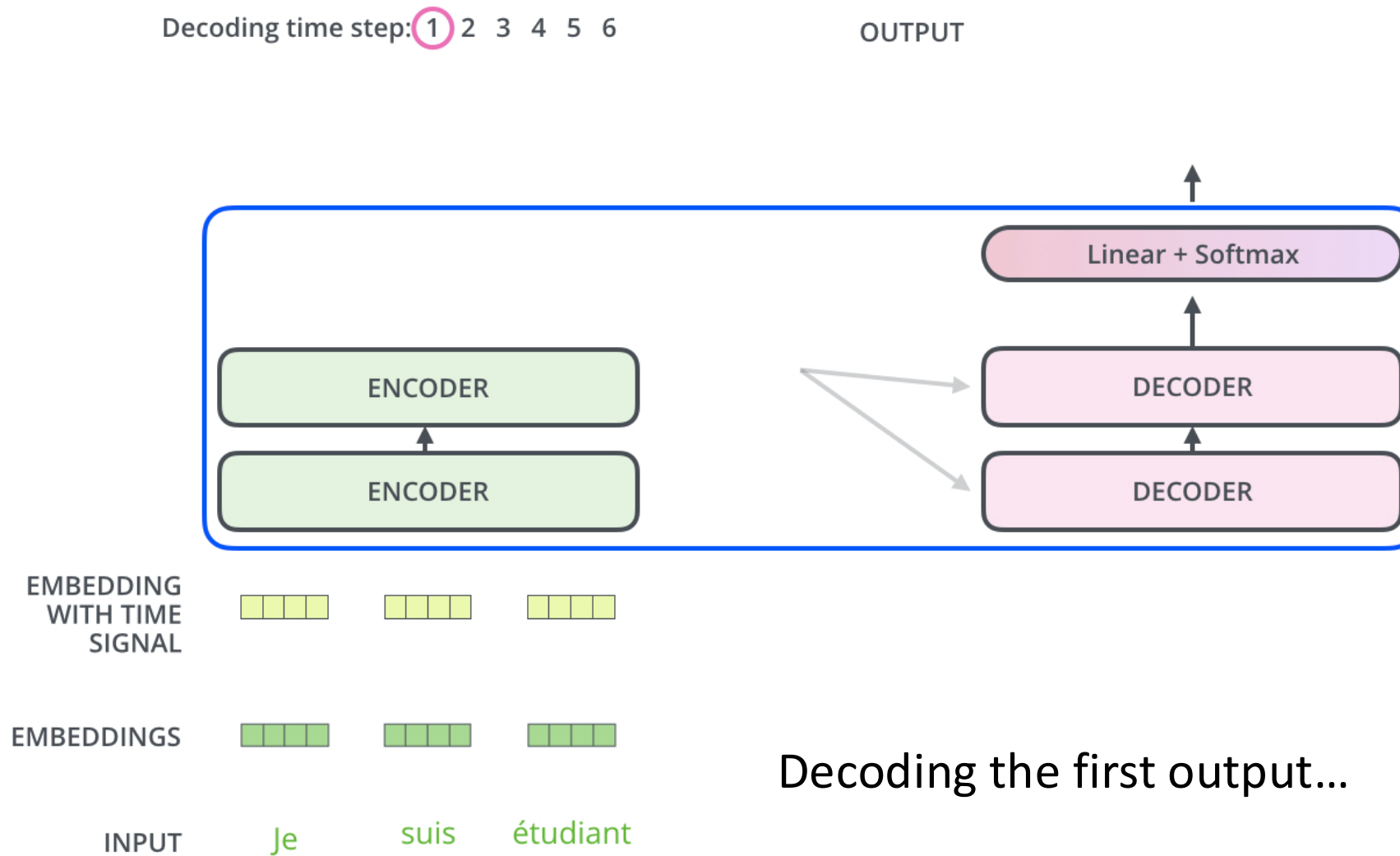


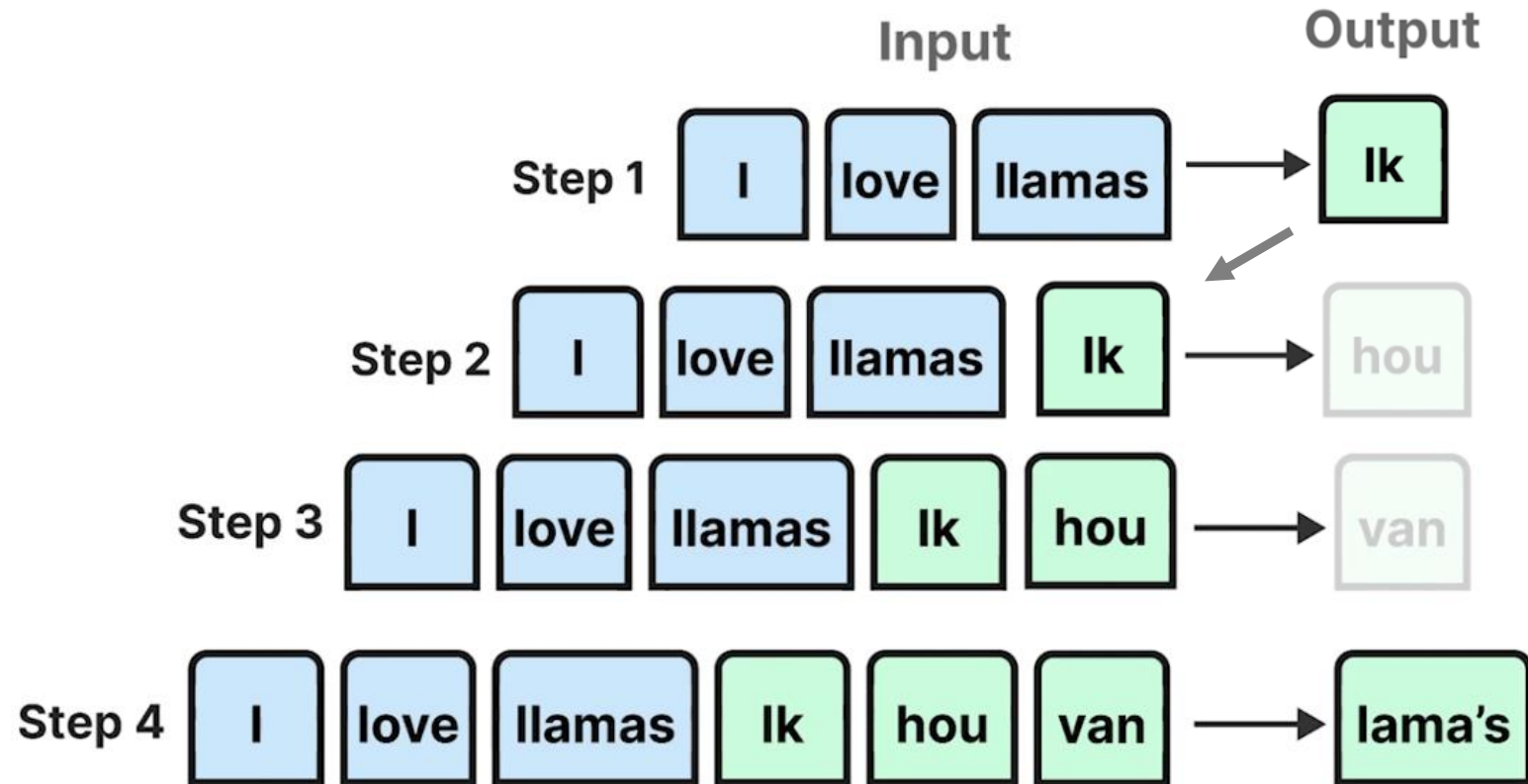
Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

What is a Transformer???



What is a Transformer???

Autoregressive ($t > 1$)



What is a Transformer???

If we zoom in...

1. Tokenization
2. Positional Encoding
3. Self-Attention
4. Output Projection

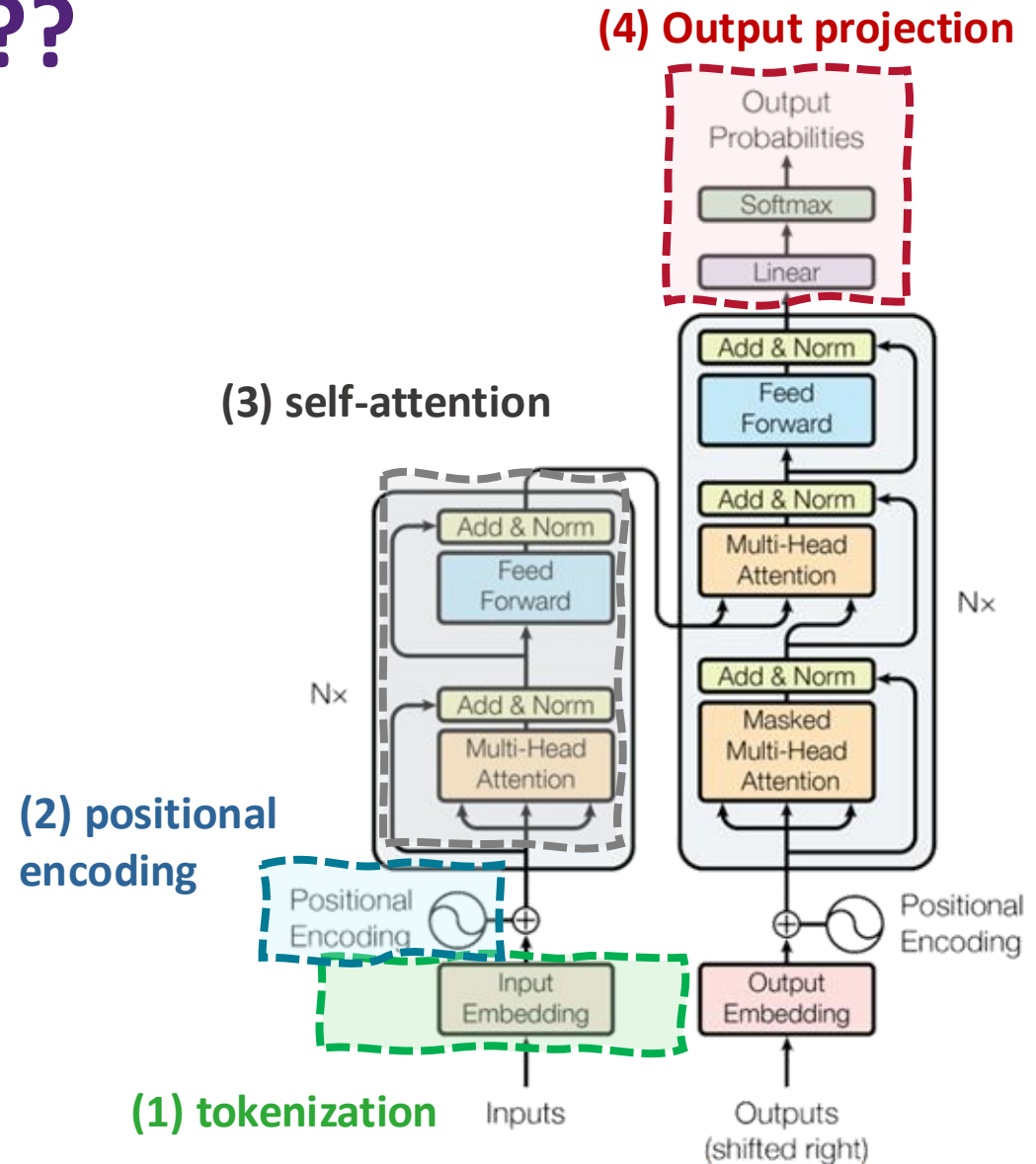
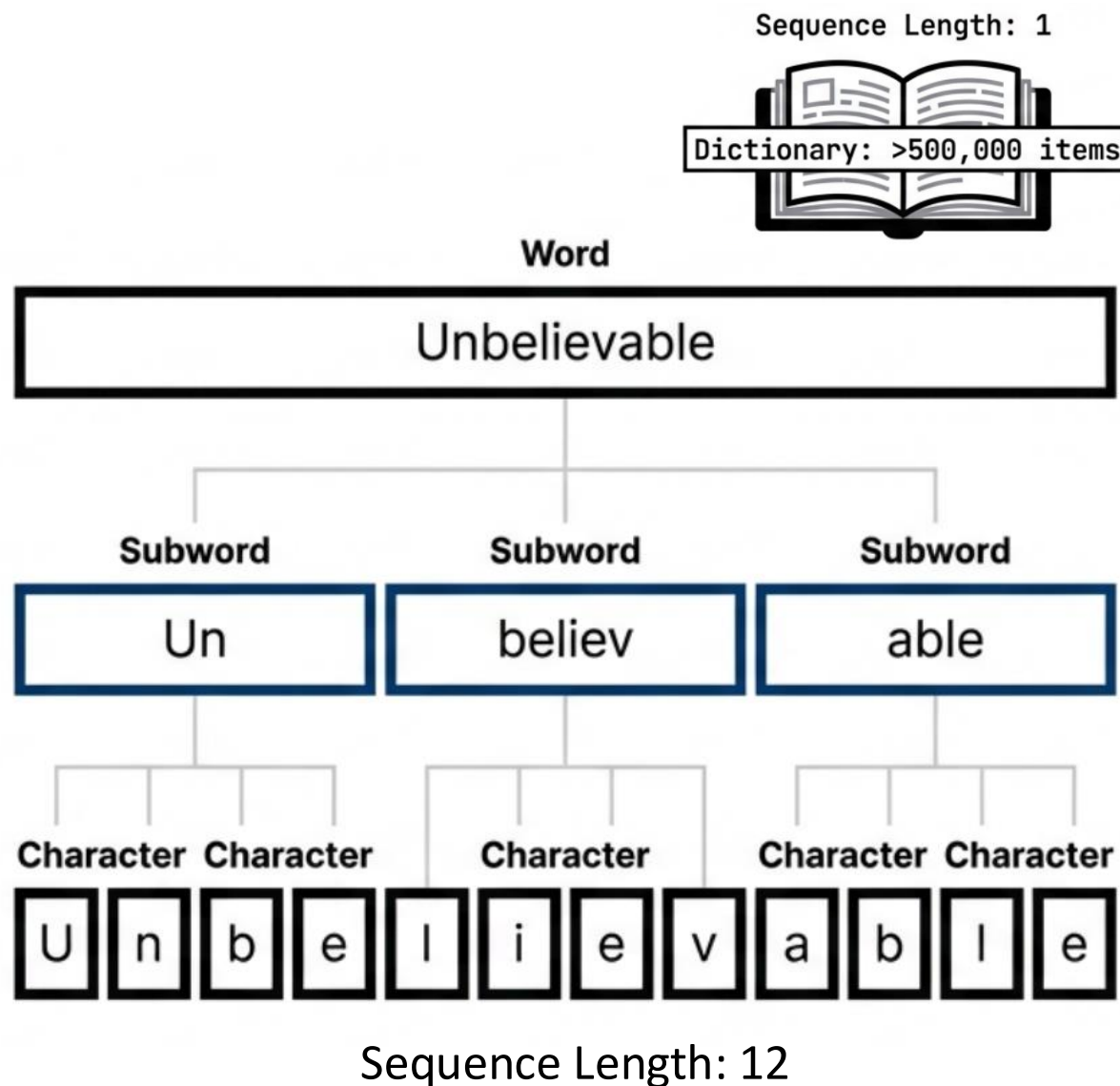


Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

1. Tokenization Basics

- **Problem:** Text is a continuous string; models need discrete inputs.
- **Solution:** Break text into atomic units called 'tokens'.
- **Granularity:** Character (too small) vs. Word (too sparse).
- **Subword:** Optimal middle ground (morphemes/stems).

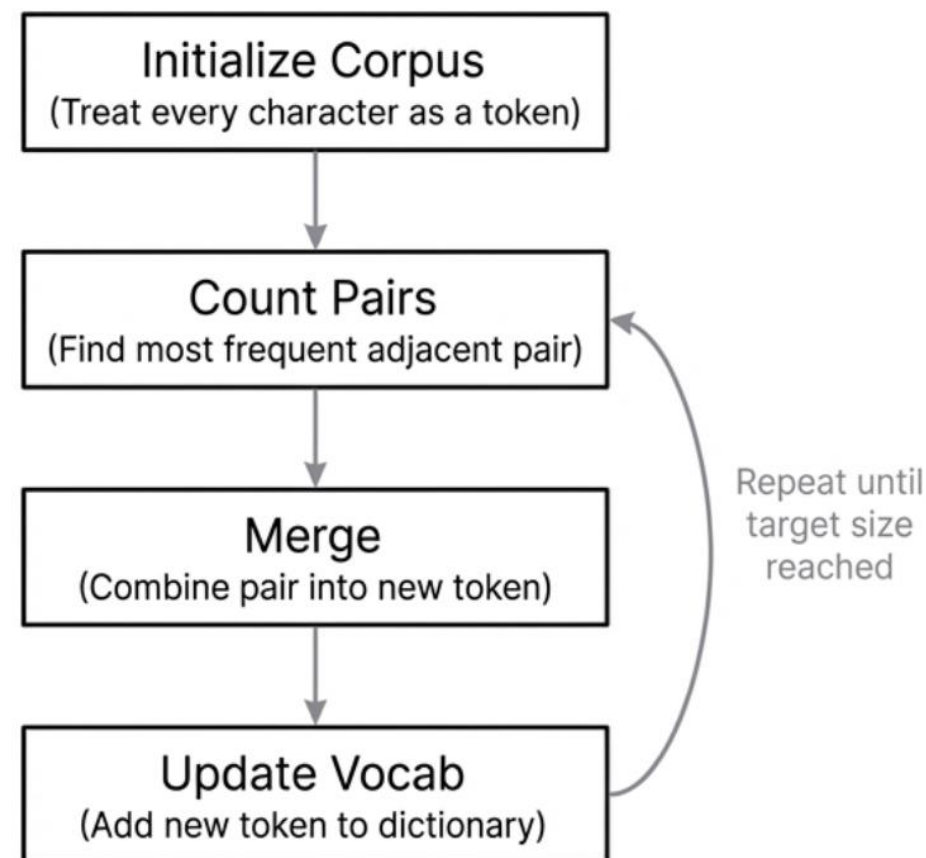
```
tokens = tokenizer.encode('Transformers are cool')  
# Output: [101, 25267, 2024, 4658, 102]
```



1. BPE (Byte-Pair Encoding)

Originally a compression algorithm (Gage, 1994), BPE is now the standard for training LLMs like GPT-4, Llama 3, and Qwen.

It iteratively merges the most frequent adjacent pairs of characters into new tokens.



1. BPE (Byte-Pair Encoding)

Step 1: The Raw Corpus & Base Vocabulary

The Data		The Initial Split	
Word	Frequency		
"hug"	10	h	u g (10)
"pug"	5	p	u g (5)
"pun"	12	p	u n (12)
"bun"	4	b	u n (4)
"hugs"	5	h	u g s (5)

Base Vocabulary: [b, g, h, n, p, s, u]

1. BPE (Byte-Pair Encoding)

Step 2: Identifying the Best Pair

Pair Frequency Analysis

u + g : 10 ("hug") + 5 ("pug") + 5 ("hugs") = **20**

u + n : 12 ("pun") + 4 ("bun") = 16

h + u : 10 ("hug") + 5 ("hugs") = 15

The Update

h **ug** (10) ← New Token Created

p **ug** (5)

p u n (12)

b u n (4)

h **ug** s (5)

1. BPE (Byte-Pair Encoding)

Step 3: Building the Vocabulary

Stage 1 (Current)

'ug' is merged.



Stage 2 (Next Merge)

'u' + 'n' appears 16 times
-> Merge to 'un'



Stage 3 (Next Merge)

'h' + 'ug' appears 15 times
-> Merge to 'hug'



[hug]

h ug p ug p u n
p ug h ug s b u n

u + n → un

p un b un h ug p ug s

h + ug → hug

hug hug s p ug p un b un

[p] [ug]

Common words become single tokens. Rare words are built from subwords.

1. BPE (Byte-Pair Encoding)

How to handle multilingual?

A	B	C	D	E	F	G
H	I	J	K	L	M	N
O	P	Q	R	S	T	U
V	W	X	Y	Z		

English Base Vocab (ASCII)						
----------------------------	--	--	--	--	--	--

大 dà big / great	小 xiǎo small	一 yī one	二 èr two	三 sān three	十 shí ten	百 bǎi hundred	千 qiān thousand	多 duō much / many	不 bù not
人 rén person	夫 fū man	子 zǐ child	男 nán male	女 nǚ female	王 wáng king	主 zhǔ master	我 wǒ I	你 nǐ you	他 tā he / she
口 kǒu mouth	说 shuō speak	目 mù eye	见 jiàn look	耳 ěr ear	闻 wén hear	手 shǒu hand	工 gōng work	脚 jiǎo foot	行 xíng go
来 lái come	入 rù enter	出 chū exit	上 shàng up / on	下 xià down / under	中 zhōng center	在 zài located at / in	左 zuǒ left	右 yòu right	有 yǒu have
吃 chī eat	饭 fàn rice (cooked)	菜 cài vegetable	豆 dòu bean	肉 ròu meat	牛 niú cow	猪 zhū pig	鸡 jī chicken	鱼 yú fish	茶 chá tea
国 guó nation	家 jiā home	校 xiào school	文 wén text	学 xué learn	狗 gǒu dog	猫 māo cat	马 mǎ horse	鸟 niǎo bird	虫 chóng insect
日 rì sun / day	月 yuè moon / month	天 tiān sky	地 dì land	海 hǎi sea	木 mù wood	火 huǒ fire	土 tǔ earth	金 jīn metal / gold	水 shuǐ water
气 qì air	干 gān dry	雨 yǔ rain	山 shān mountain	川 chuān river	米 mǐ rice (grain)	田 tián field	花 huā flower	石 shí stone	玉 yù jade
村 cūn village	店 diàn shop	车 chē vehicle	衣 yī clothes	白 bái white	黑 hēi black	红 hóng red	蓝 lán blue	绿 lǜ green	黄 huáng yellow
是 shì be	心 xīn heart / mind	好 hǎo good	爱 ài love	喜 xǐ happiness	幸 xìng luck	生 shēng life	死 sǐ death	力 lì strength	病 bìng sickness

Modern Dictionary:
Typically lists **12,000 to 20,000** characters in somewhat regular use

1. Byte-level BPE

Using UTF-8 Bytes as the Universal Base



No More Unknowns. Instead of defining a vocabulary of characters, modern models (GPT-4, Qwen) define a vocabulary of **Bytes** (in **Syntax Blue**).

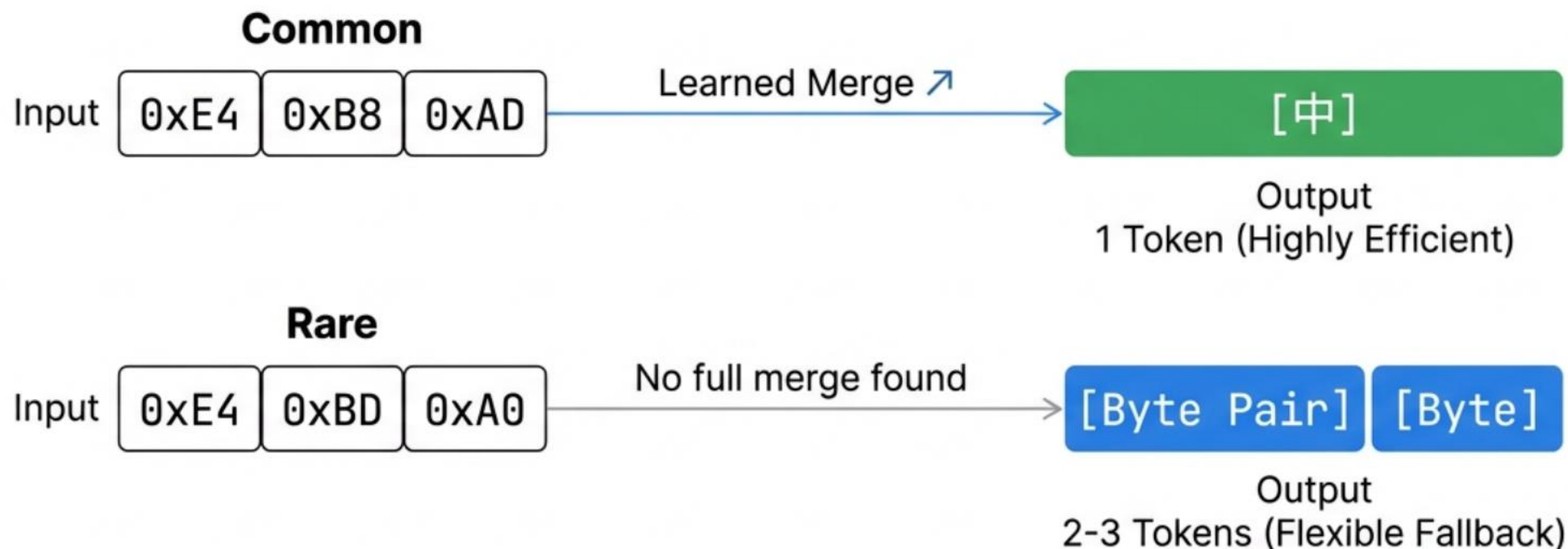
Base Vocabulary 256 possible byte values.

Any text—Chinese, Emoji, Code—can be represented as a sequence of bytes.

BPE then learns to merge these bytes back into characters and words.

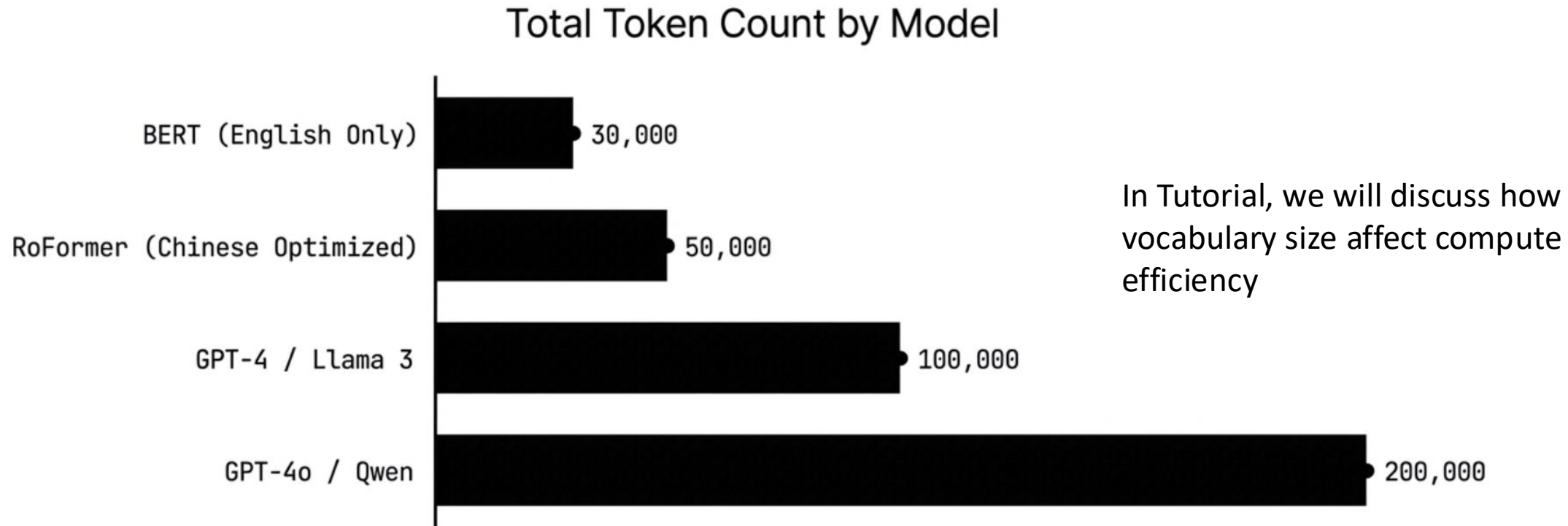
1. Byte-level BPE

Efficiently merging bytes for multilingual mastery



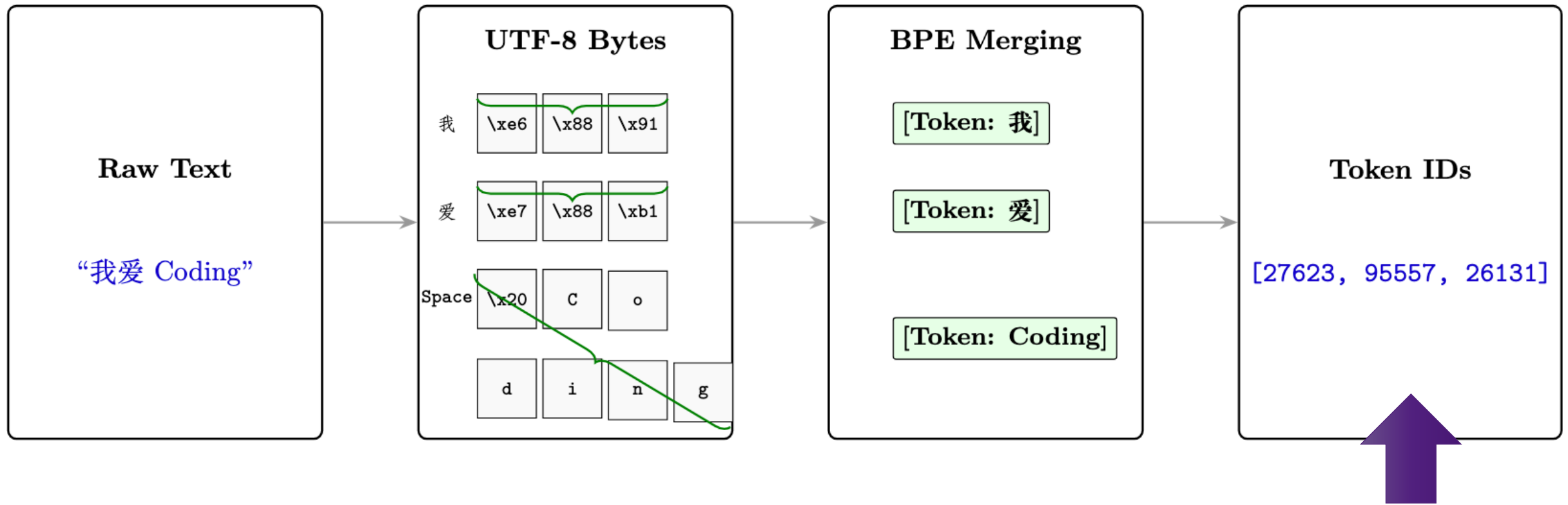
Qwen uses a large vocabulary to ensure that common Chinese words are compressed into single tokens, just like English words.

1. The Cost of Multilingualism



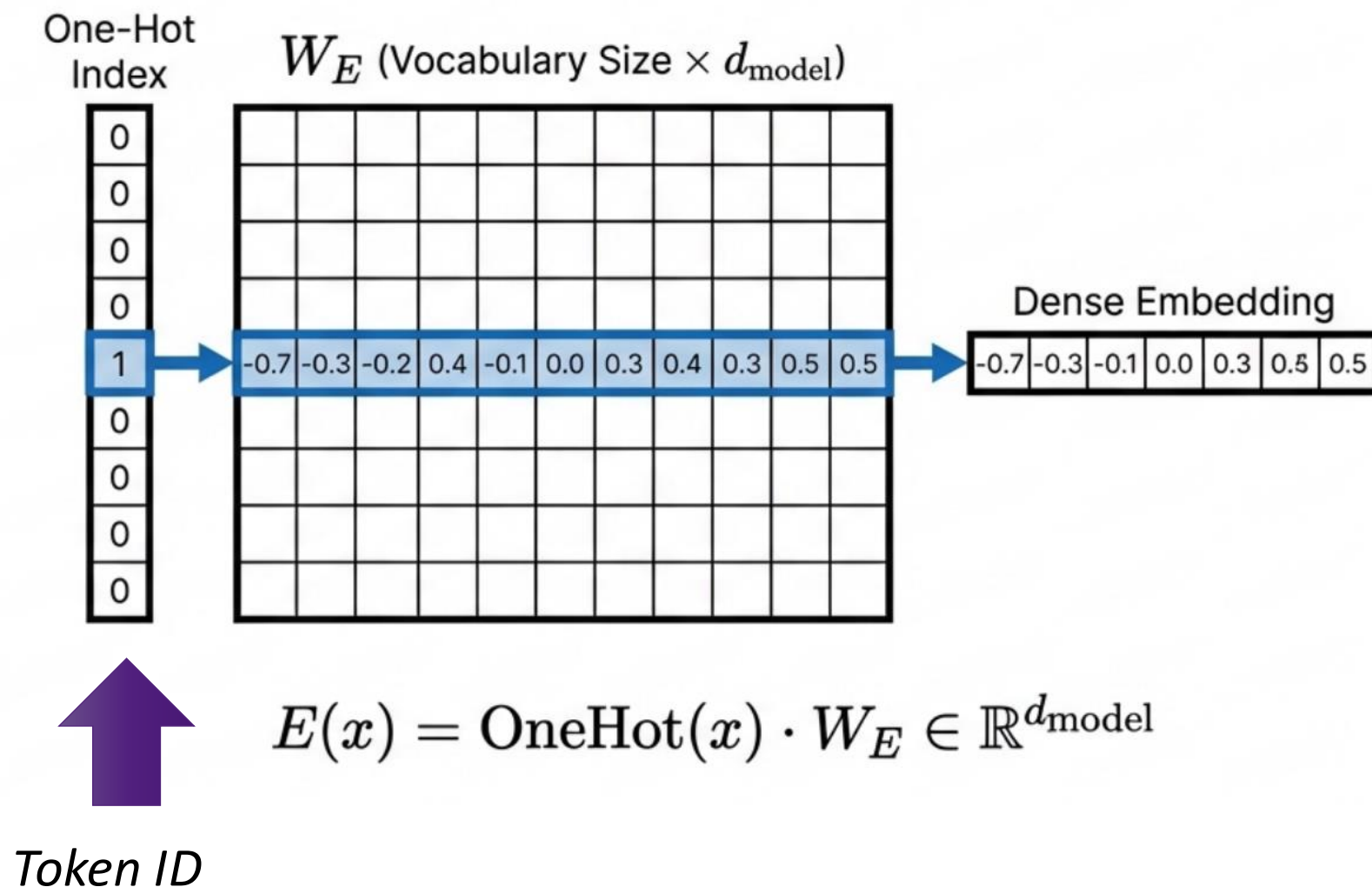
Larger vocabularies allow “sparse” coverage of many languages, giving each language its own dedicated tokens.

1. Byte-level BPE



1. Embedding Lookups

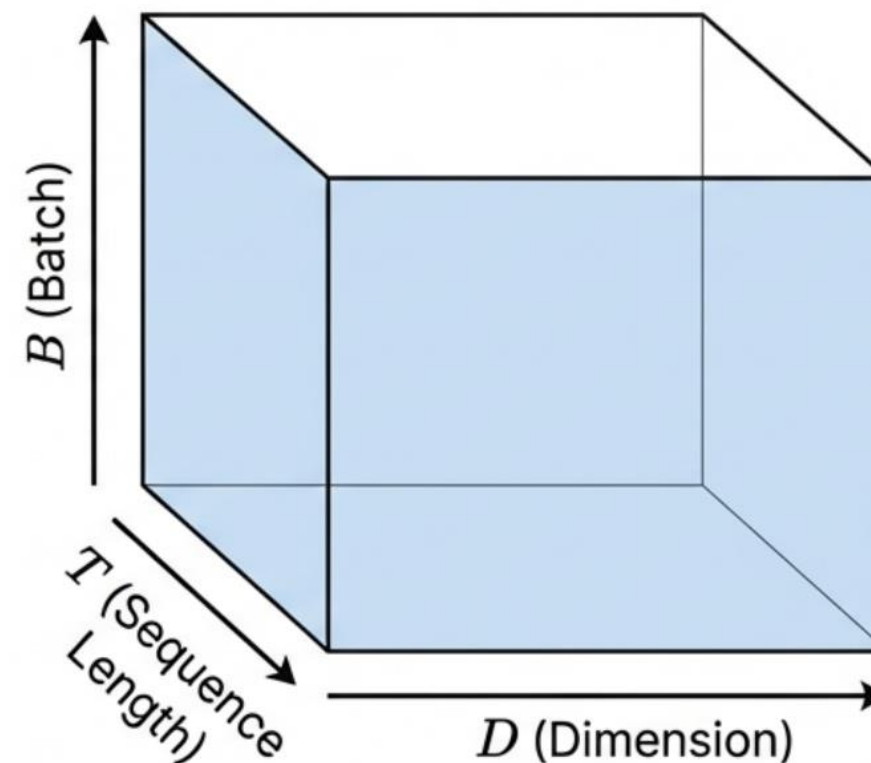
- **Input:** Integer indices (Token IDs).
- **Operation:** Select row from a learnable weight matrix W_E .
- **Nature:** Dense, continuous vector representation.
- **Semantics:** Initially random, learns meaning via backpropagation.



1. Embedding Tensors

- **Batch** (B): Number of sequences processed in parallel.
- **Sequence** (T): Number of tokens (time steps).
- **Dimension** (D): Hidden size (feature depth).
- **Flow**: Shape remains (B, T, D) throughout the Encoder.

```
x = embedding(input_ids)
print(x.shape) # torch.Size([32, 128, 768])
```

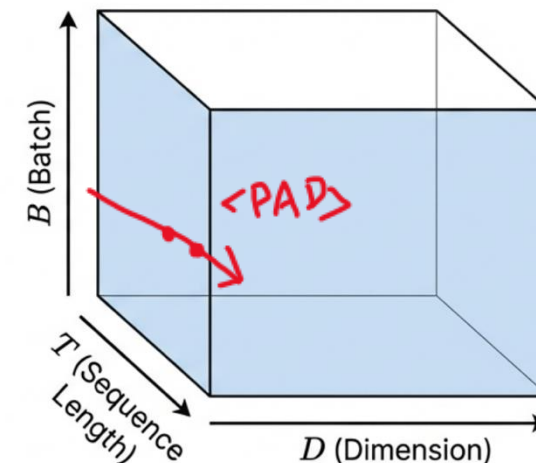


*What if we have sequence of **different** lengths?*

1. Embedding Tensors

- **Reality:** Batches contain sequences of different lengths.
- **Pad Tokens:** Fillers added to match the longest sequence.
- **Masking:** Prevent Attention from 'seeing' pad tokens.
- **Implementation:** Add $-\infty$ to attention scores before Softmax.

	Key Tokens			Pad Token ($-\infty$)
Query Tokens	0.1	0.5	0.2	
	0.3	0.4	0.1	
	0.6	0.2	0.5	
	0.2	0.3	0.4	
				Pad Token ($-\infty$)



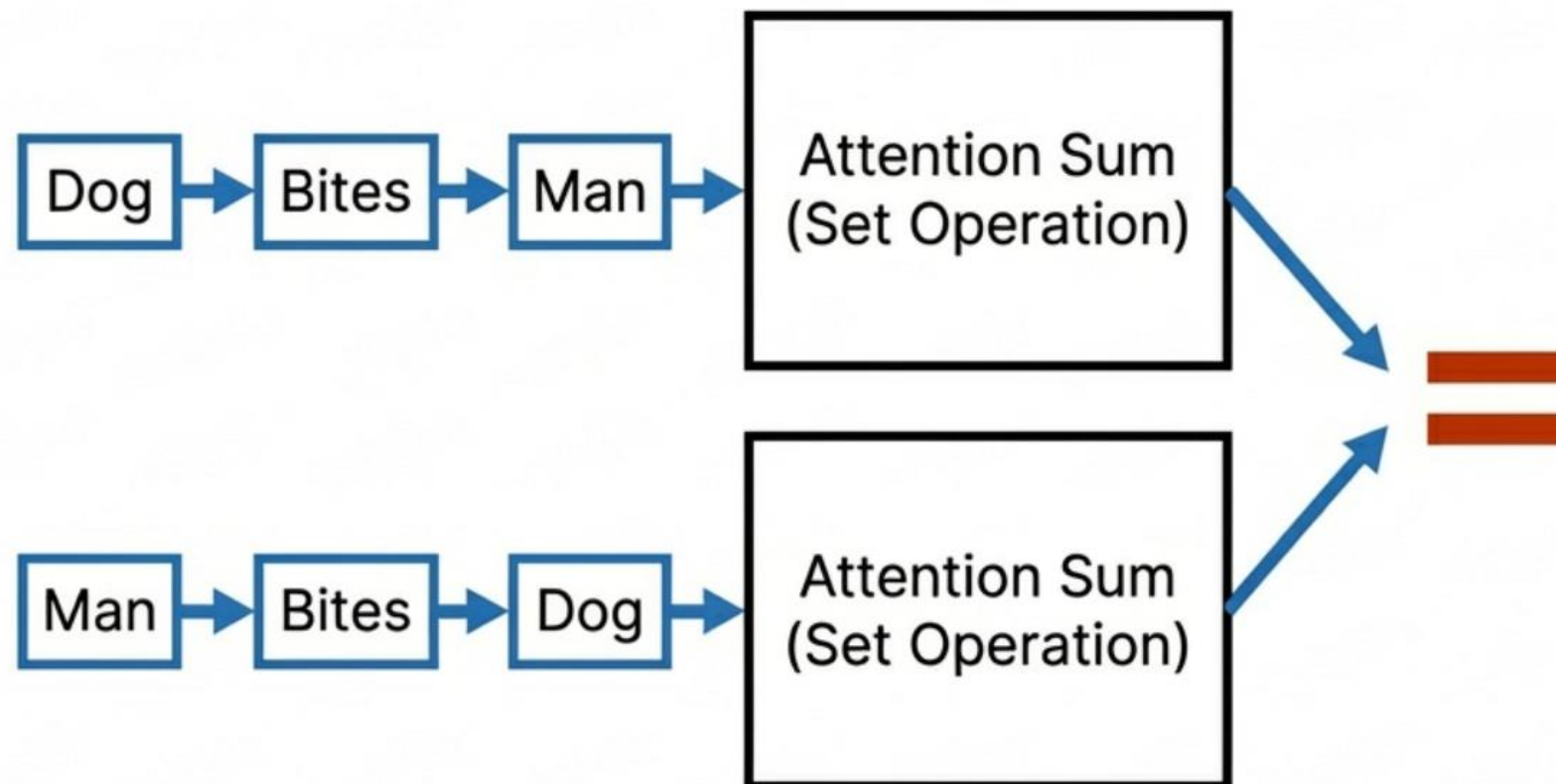
$$A_{ij} = \text{softmax}(S_{ij} + M_{ij}) \quad M_{ij} \in \{0, -\infty\}$$

1. Tokenization

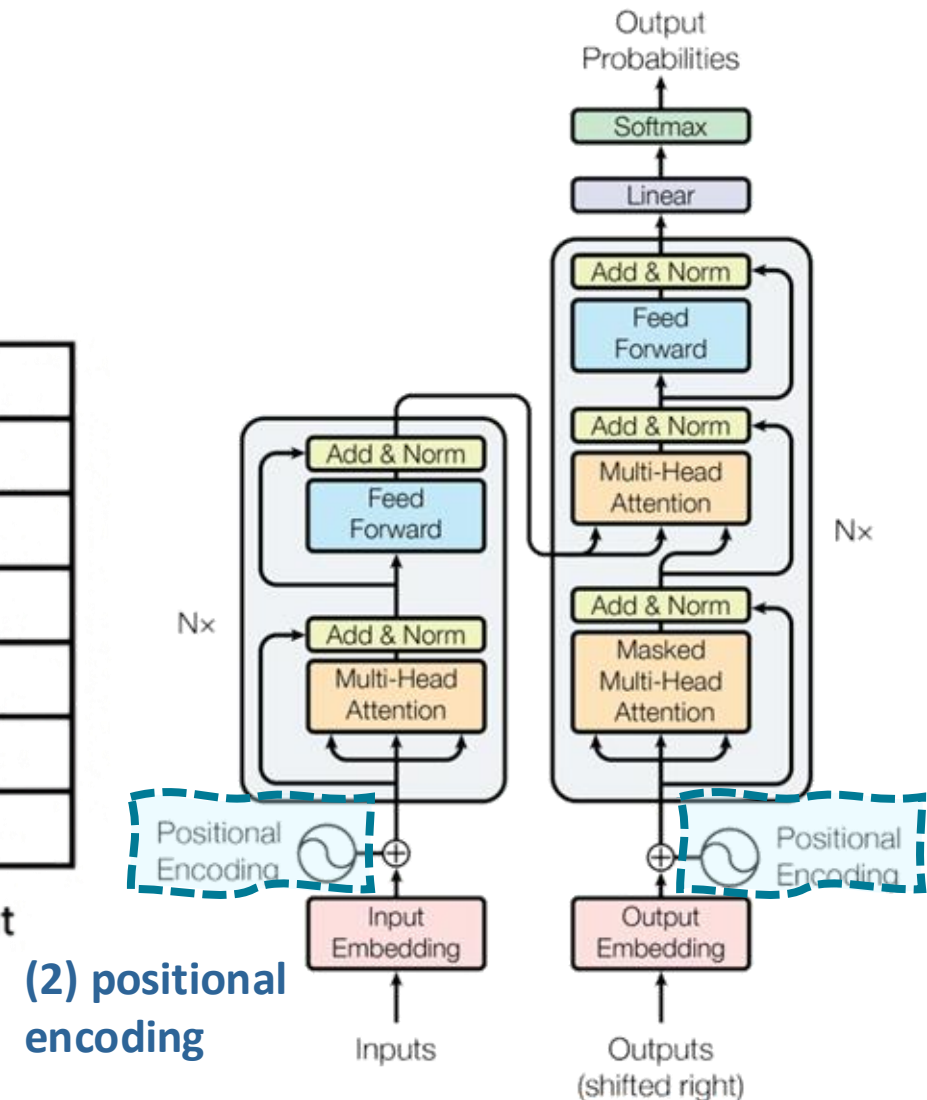
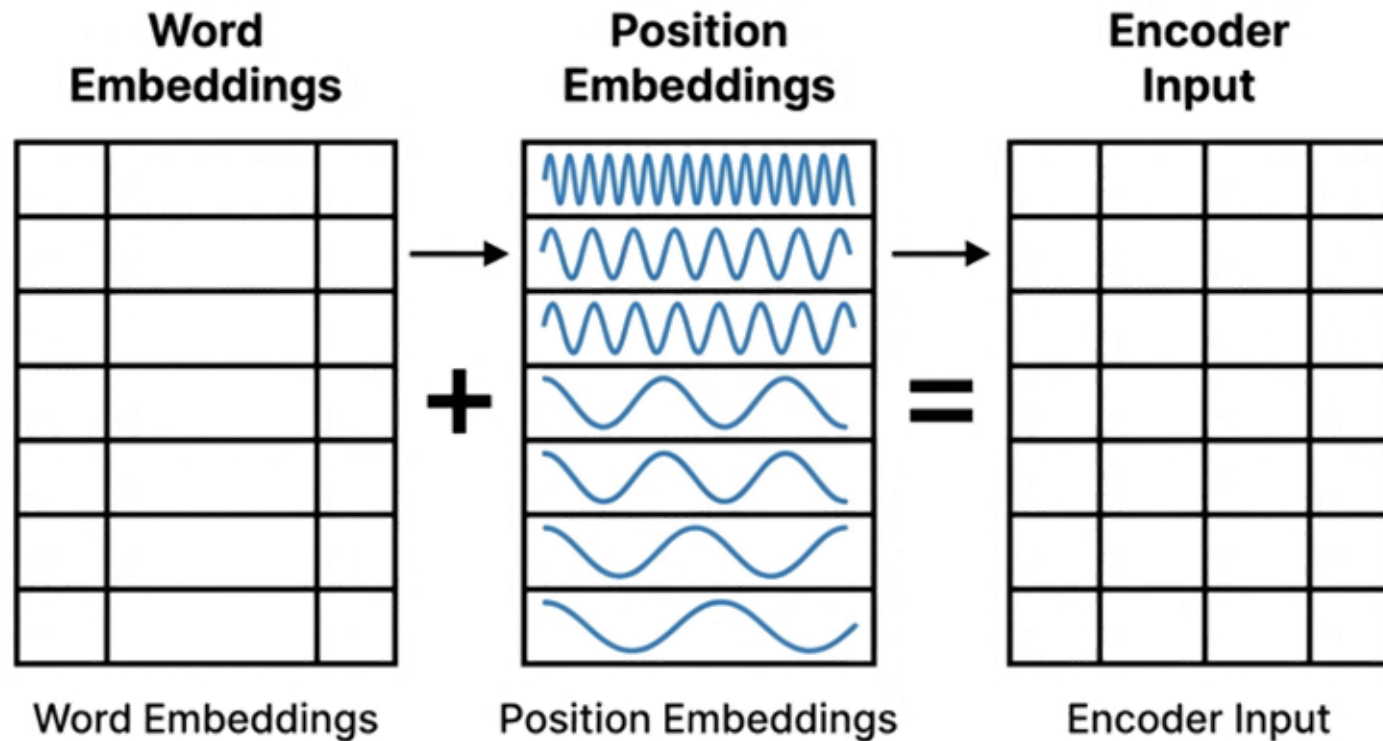
So far, our embedding is still permutation-invariant...



- **Issue:** Self-Attention is a set operation, not a sequence operation.
- **Implication:** Swapping tokens x_1 and x_2 yields identical featural features.
- **Context:** “Dog bites man” $\neq$ “Man bites dog”.



2. Positional Encoding



(2) positional encoding

Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

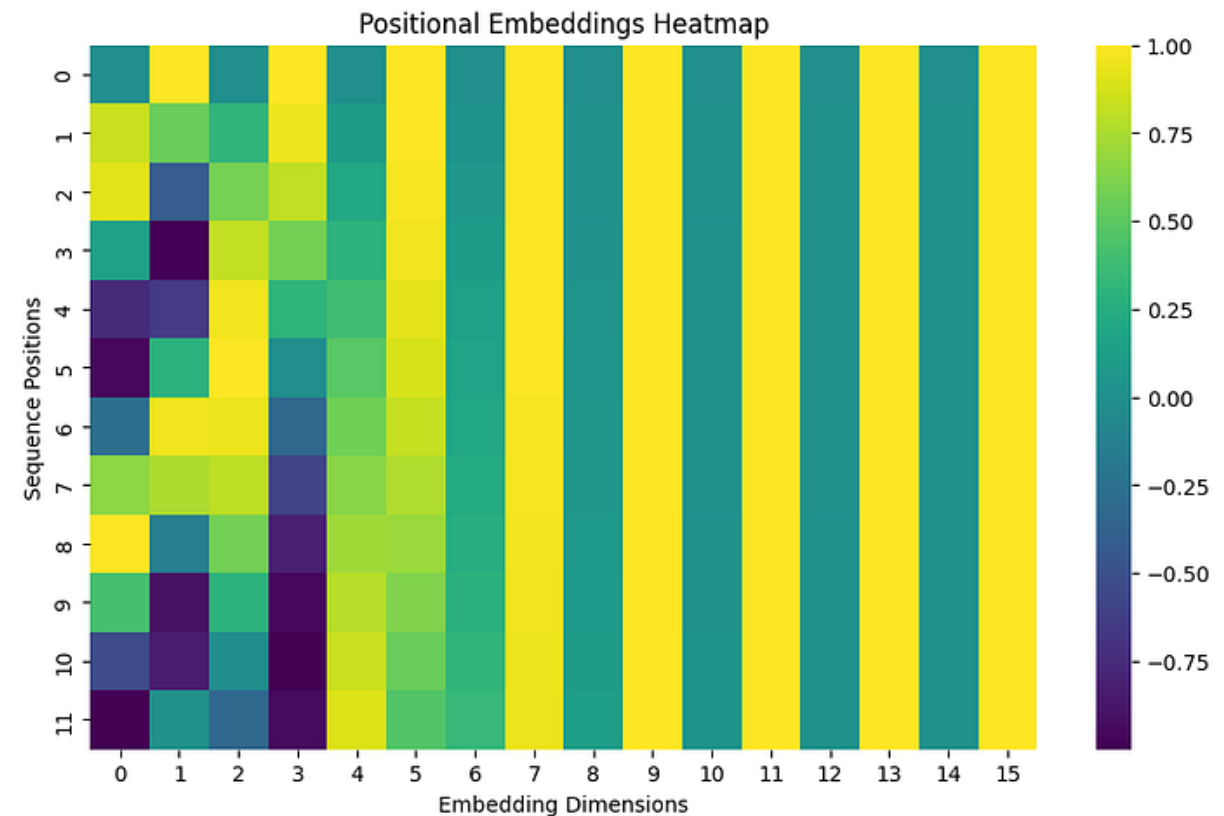
2. Positional Encoding

Absolute Positional Embedding (PE)

- Learned embeddings for every position;

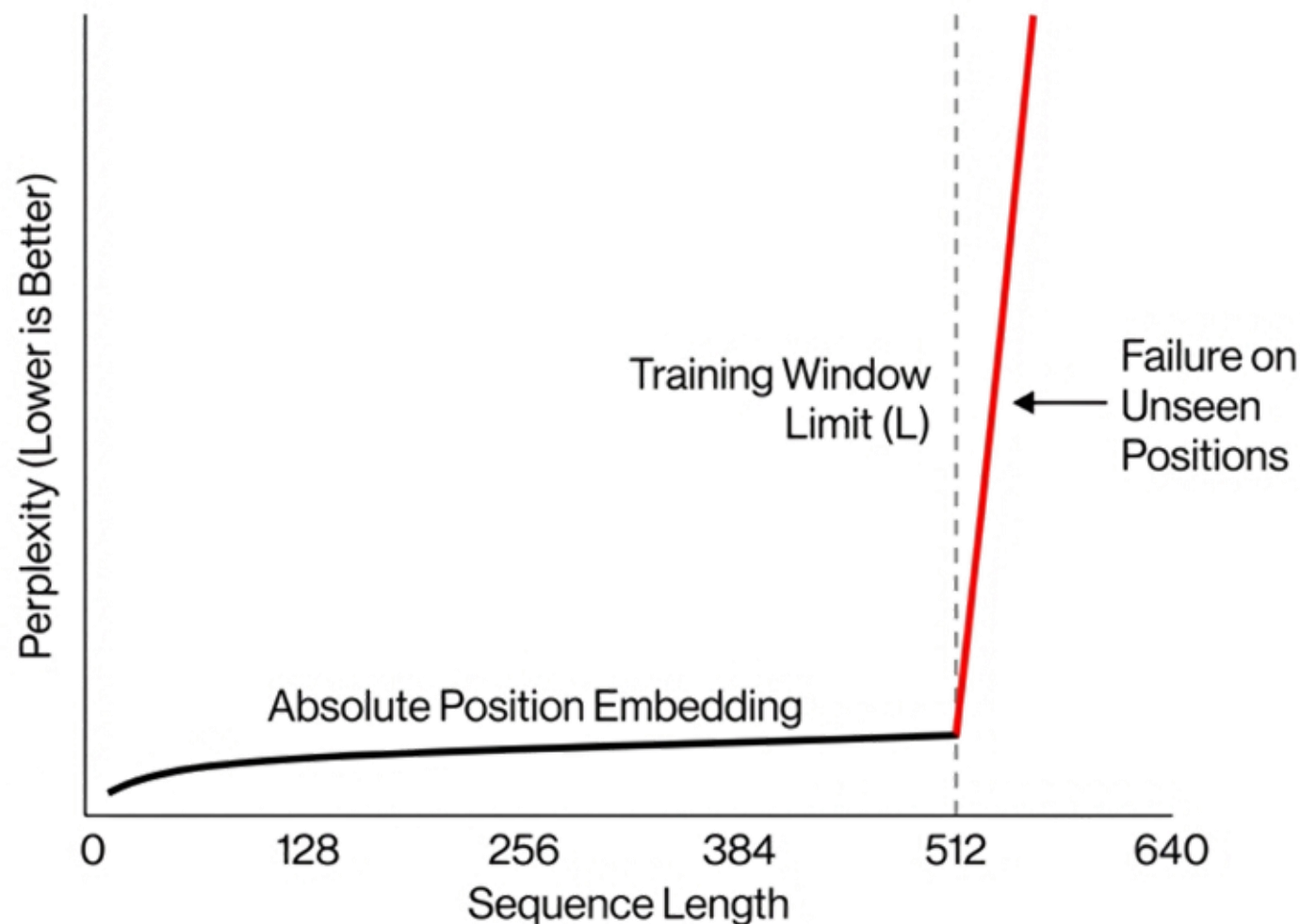
```
import torch.nn as nn
class LearnedPositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=512):
        super().__init__()
        self.positional_embedding = nn.Embedding(max_len, d_model)
    def forward(self, x):
        positions = torch.arange(x.size(1), device=x.device).unsqueeze(0)
        return self.positional_embedding(positions)
```

$$x_p = x + \text{PE}(p)$$



2. Positional Encoding

The Extrapolation Problem...



- **Limitation:** Models with Absolute PE memorize specific indices (0 to 512).
- **Failure Mode:** At index 513, the model encounters an undefined embedding vector.
- **Result:** “Train short, test long” is impossible without a relative mechanism.

2. Positional Encoding

Absolute Positional Embedding (PE)

- Learned embeddings for every position; or
- Sinusoid of different frequencies

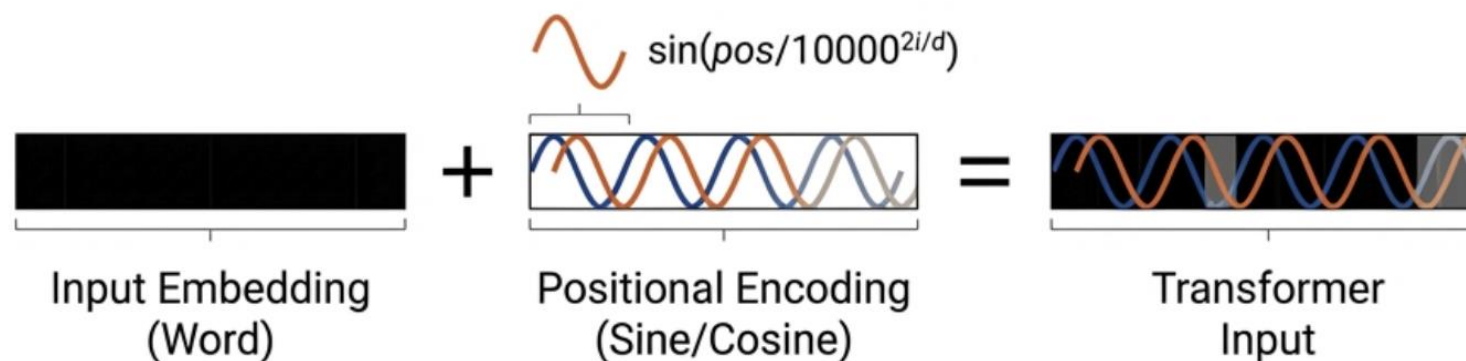
pos : Absolute position (0, 1, 2...)

i : Dimension index (0 to $d_{model}/2$)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

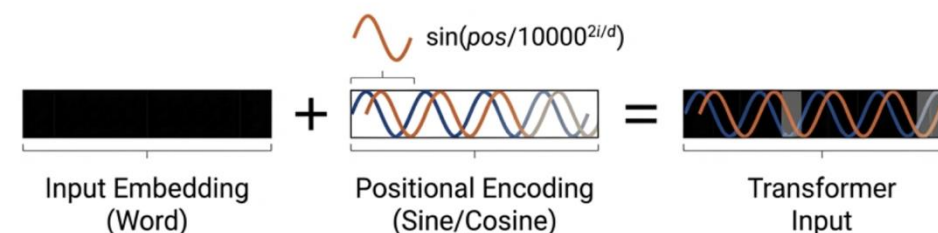
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

How does sine and cosine function represent positions?



2. Positional Encoding

Case study: when $d=4$;



Calculating the position vector for Position 1 ($pos=1$).

Dimension

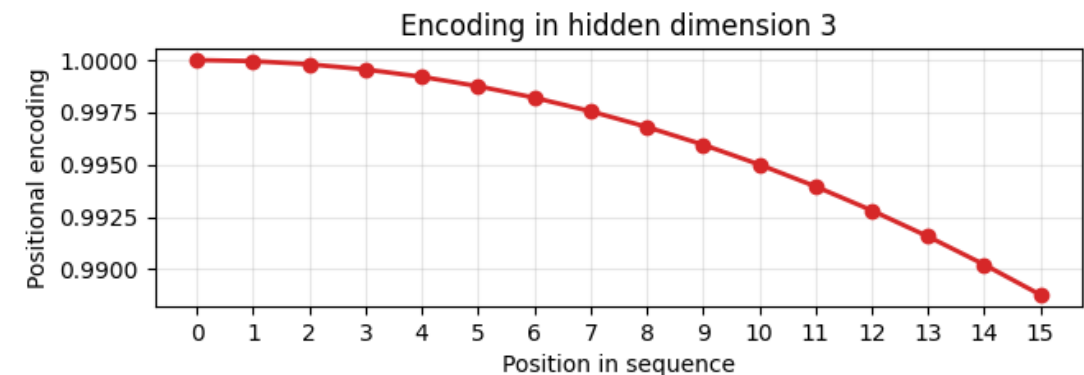
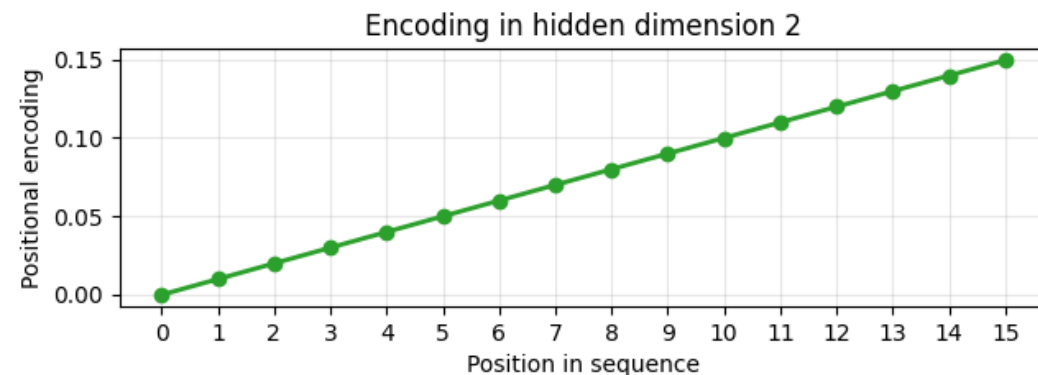
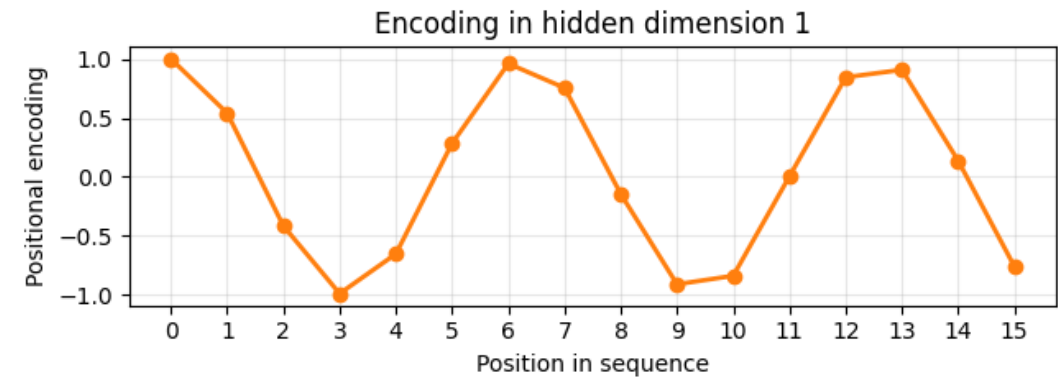
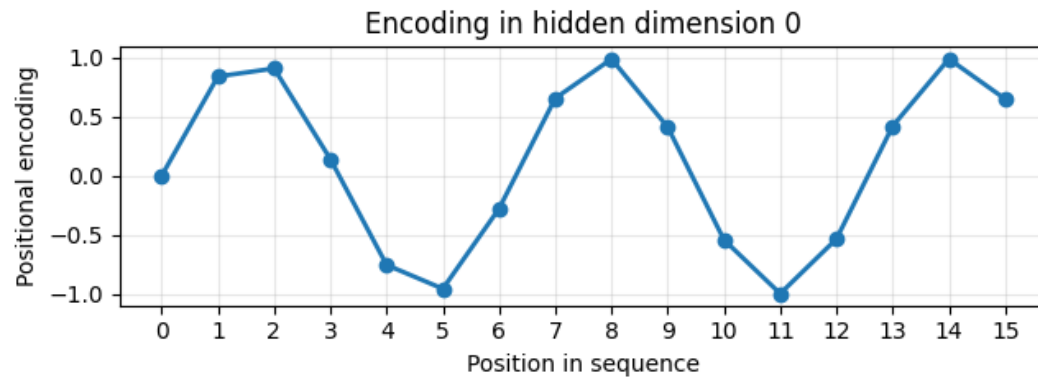
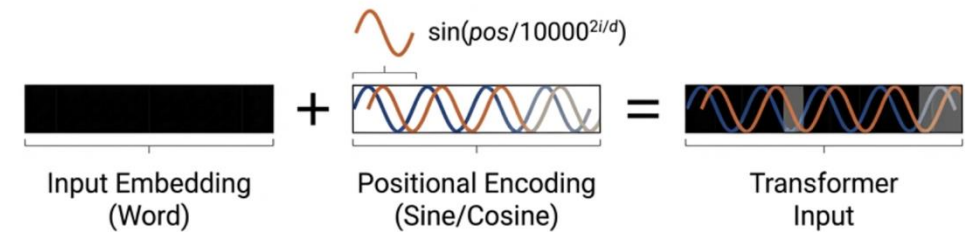
(the second token in the sequence)

Index 0 (Even)	$\sin(1/10000^{0/4})$	Result: $\sin(1) \approx 0.84$	
Index 1 (Odd)	$\cos(1/10000^{0/4})$	Result: $\cos(1) \approx 0.54$	
Index 2 (Even)	$\sin(1/10000^{2/4})$	Result: $\sin(1/100) \approx 0.01$	
Index 3 (Odd)	$\cos(1/10000^{2/4})$	Result: $\cos(1/100) \approx 1.00$	

Final Added PE Vector: [0.84, 0.54, 0.01, 1.00]

2. Positional Encoding

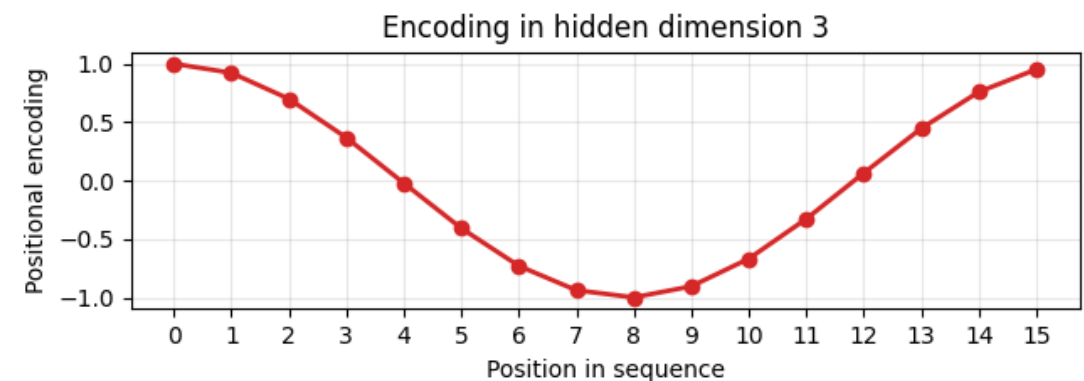
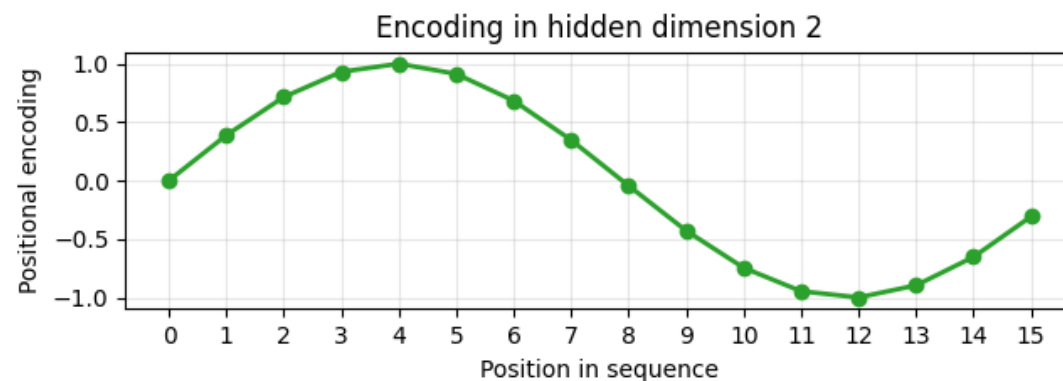
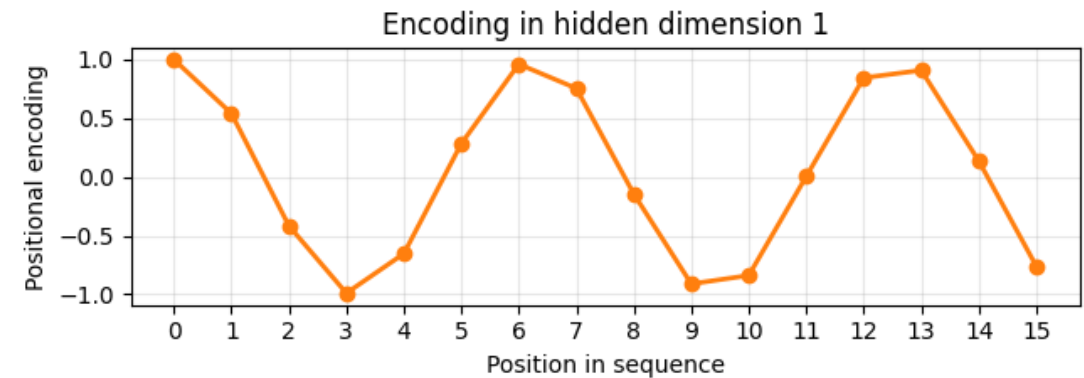
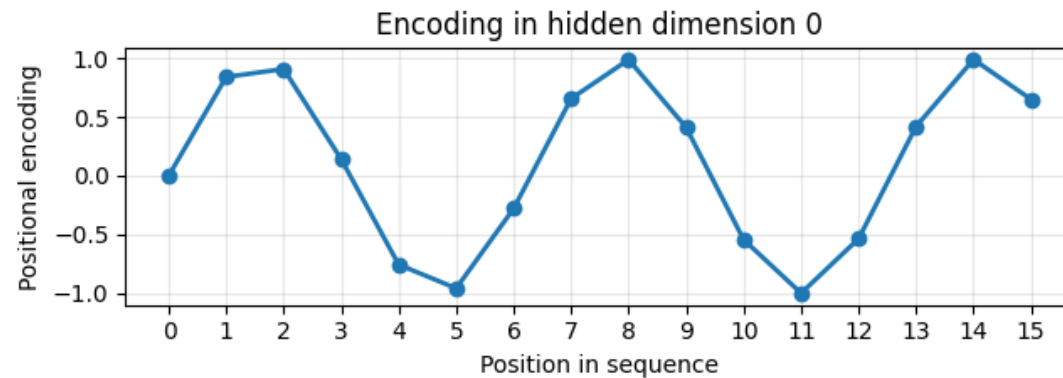
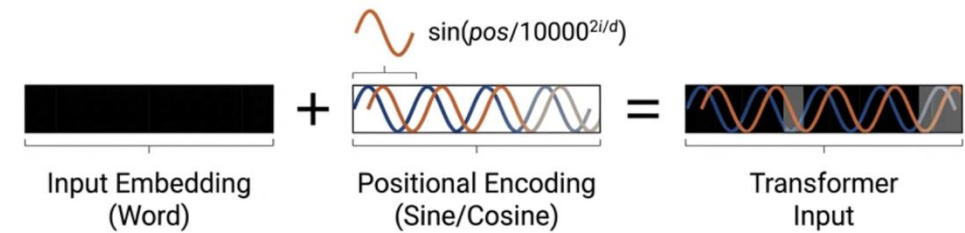
Case study: when $d=4$;



What's the pattern?

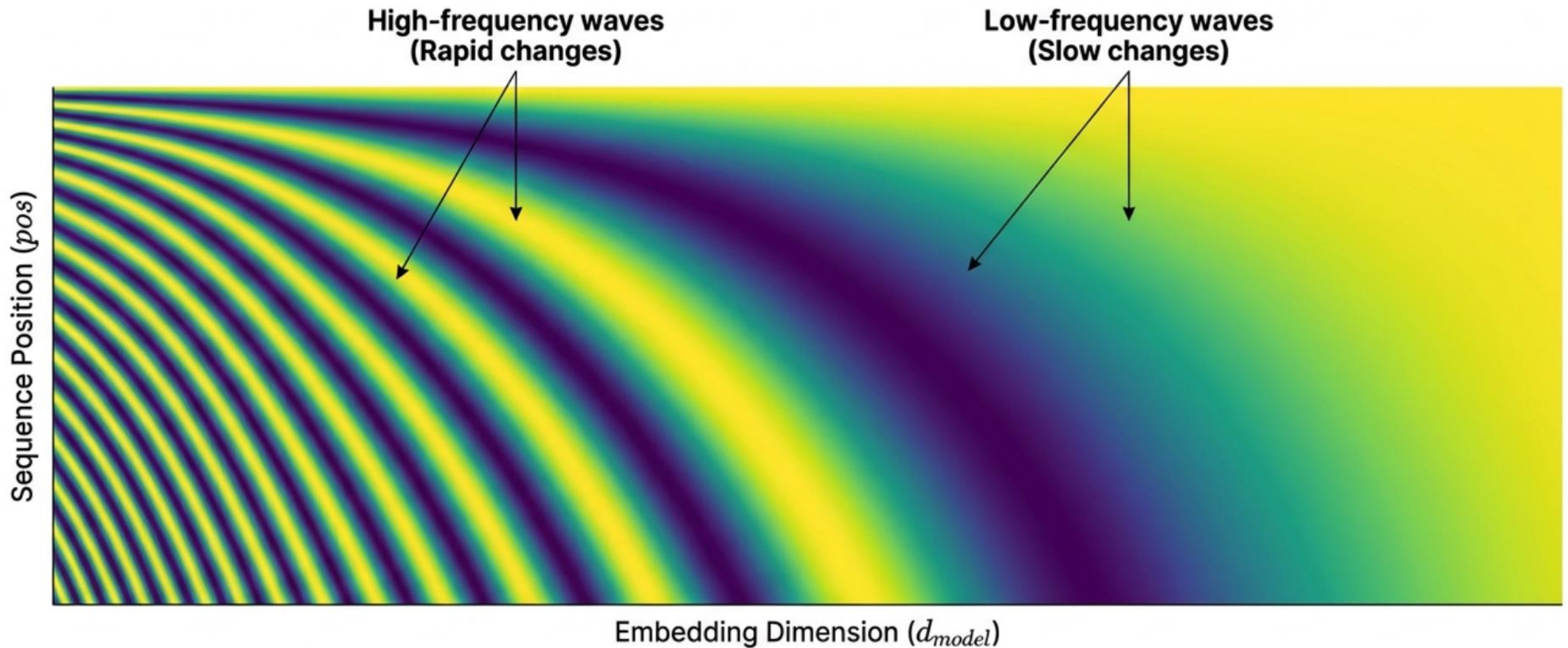
2. Positional Encoding

Case study: when $d=20$;

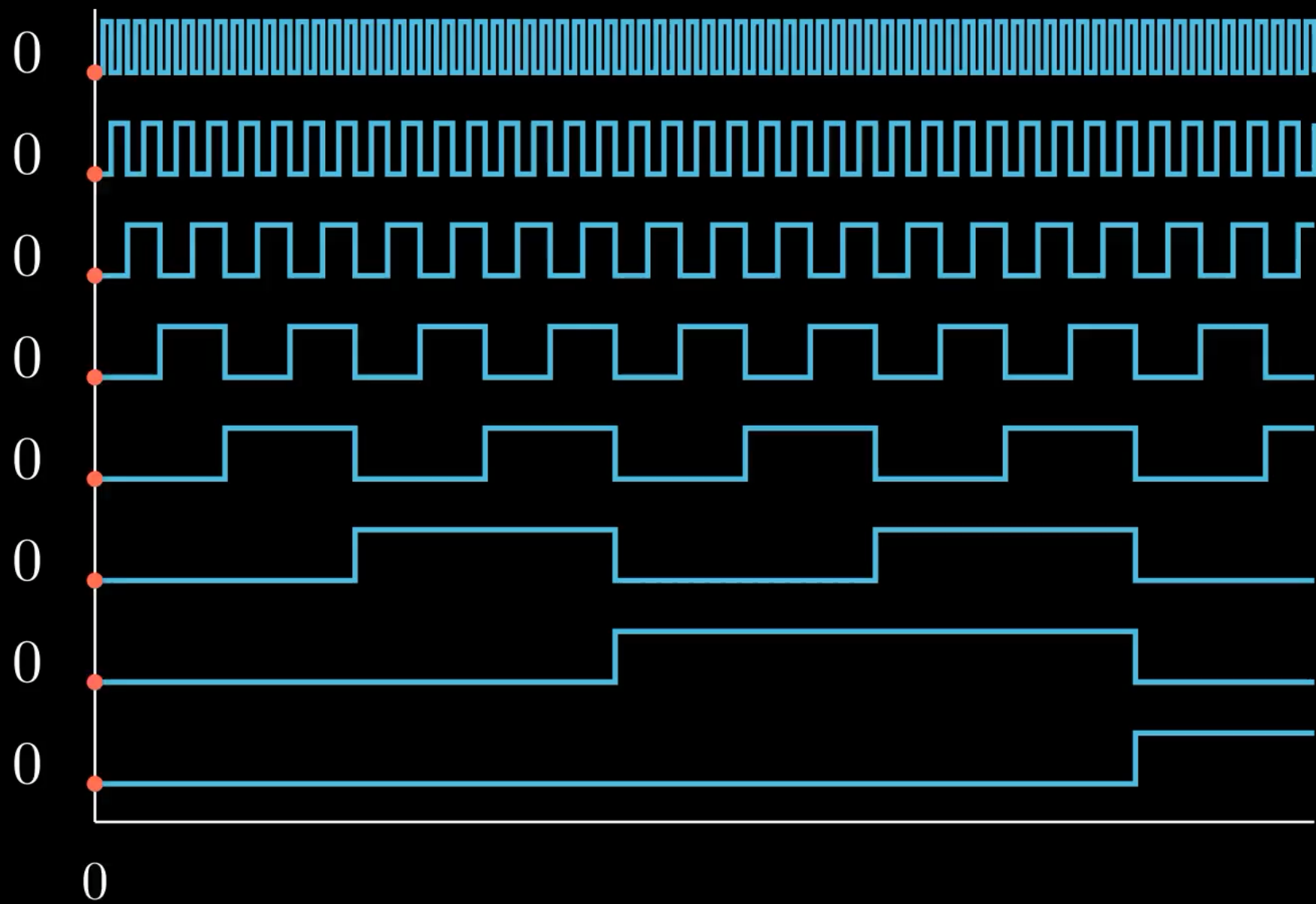


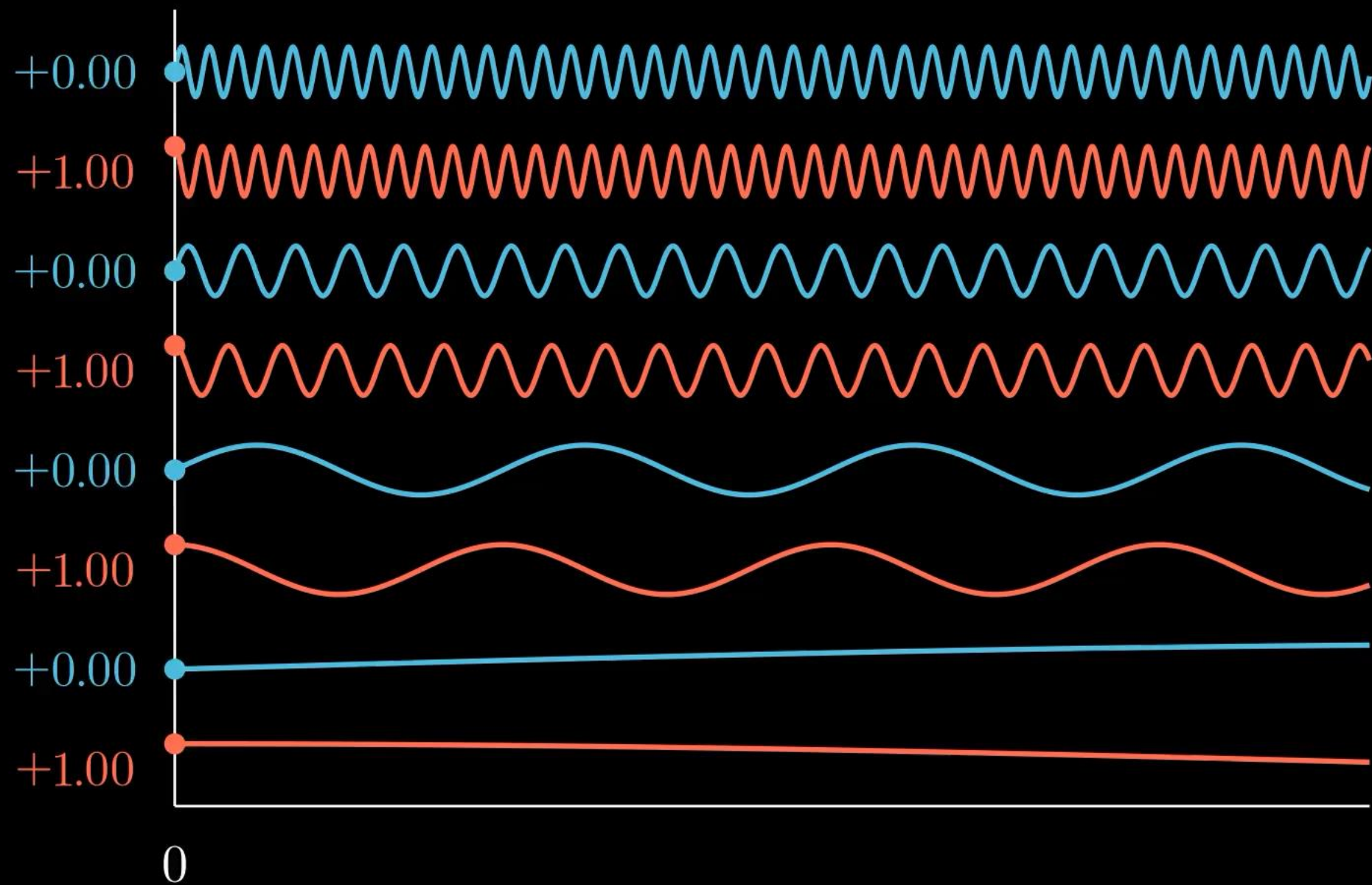
What's the pattern?

2. Positional Encoding



Analogy: Like a clock. The 'seconds' hand (low dimension) moves fast; the 'hours' hand (high dimension) moves slow. The combination uniquely identifies the timestamp.





$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$



Let's have a Quiz!

2. Positional Encoding

Why use Sinusoids? $x_p = x + \text{PE}(p)$

The Linear Interaction Property

This choice of functions should allow the model to attend to relative positions, since for any fixed k , PE_{pos+k} is a linear function of PE_{pos} , because

$$\text{PE}_{pos} \quad \text{PE}_{pos+k}$$

$$\mathbf{M} \cdot \begin{bmatrix} \sin(\omega_i p) \\ \cos(\omega_i p) \end{bmatrix} = \begin{bmatrix} \sin(\omega_i (p + k)) \\ \cos(\omega_i (p + k)) \end{bmatrix} \quad \omega_i = \frac{1}{10000^{2i/d}}$$

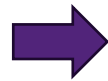
2. Positional Encoding

Why use Sinusoids? $x_p = x + \text{PE}(p)$

The Linear Interaction Property

This choice of functions should allow the model to attend to relative positions, since for any fixed k , PE_{pos+k} is a linear function of PE_{pos} , because

$$\begin{bmatrix} \sin(\omega_i(p+k)) \\ \cos(\omega_i(p+k)) \end{bmatrix} = \begin{bmatrix} \sin(\omega_i p) \cos(\omega_i k) + \cos(\omega_i p) \sin(\omega_i k) \\ \cos(\omega_i p) \cos(\omega_i k) - \sin(\omega_i p) \sin(\omega_i k) \end{bmatrix}$$



$$\sin(a+b) = \sin a \cos b + \cos a \sin b$$

$$\mathbf{M}_k \text{PE}_{pos} = \text{PE}_{pos+k}$$

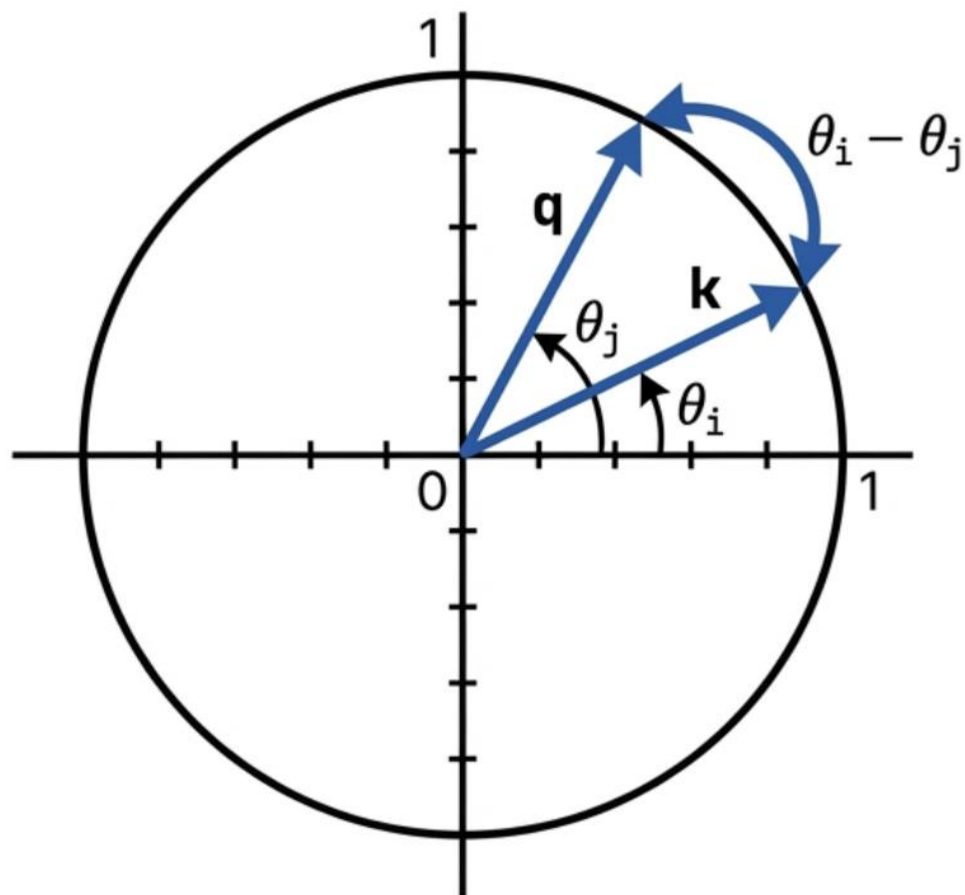


$$\mathbf{M}_k = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix}$$

Transformation Matrix

(Rotation Matrix)

2. Positional Encoding



Rotary Position Embedding (ROPE)

$$\begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}$$

$$(R(\theta_i)\mathbf{q}) \cdot (R(\theta_j)\mathbf{k}) = \mathbf{q} \cdot R(\theta_i - \theta_j)\mathbf{k}$$

- Translation Invariance: The attention score depends **only** on the relative distance (angle difference) between tokens.
- Result: The interaction between tokens at positions 5 and 10 is mathematically identical to tokens at 105 and 110.

3. Self-Attention

*If we zoom in * 2...*

1. Multi-head Attention
2. Norm
3. Feedforward

(3) self-attention

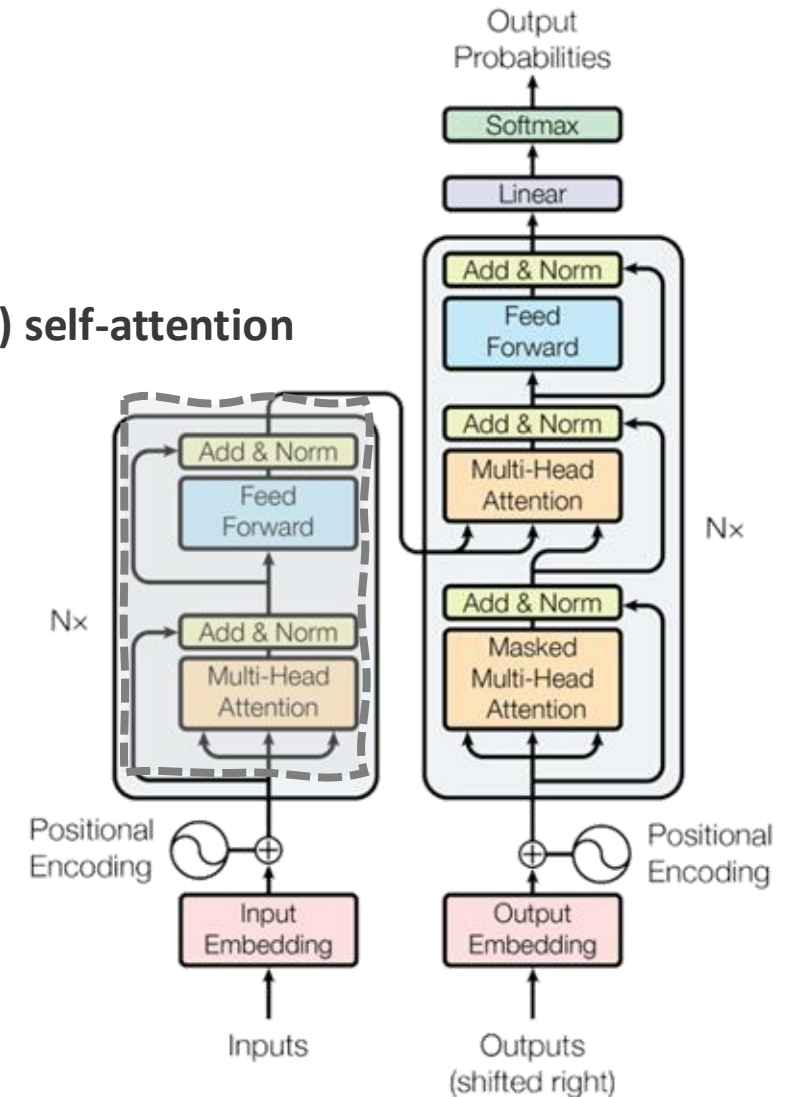


Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

3. Self-Attention: Scaled Dot-Product

Assume that we have a sequence of n words represented using a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$.

The attention module for a queries $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{n \times d_k}$ and values $\mathbf{V} \in \mathbb{R}^{n \times d}$ defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}.$$

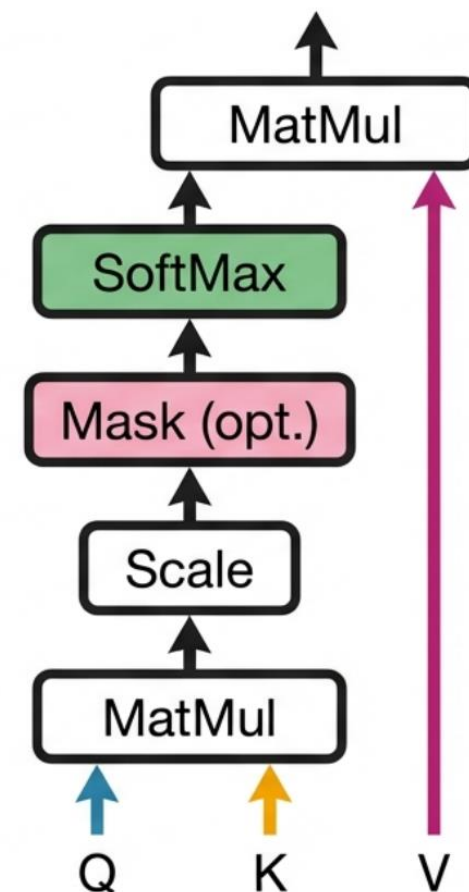
The queries, keys and values are computed from the input word representations $\mathbf{X}$ using a linear transformation as

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^K$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^V$$

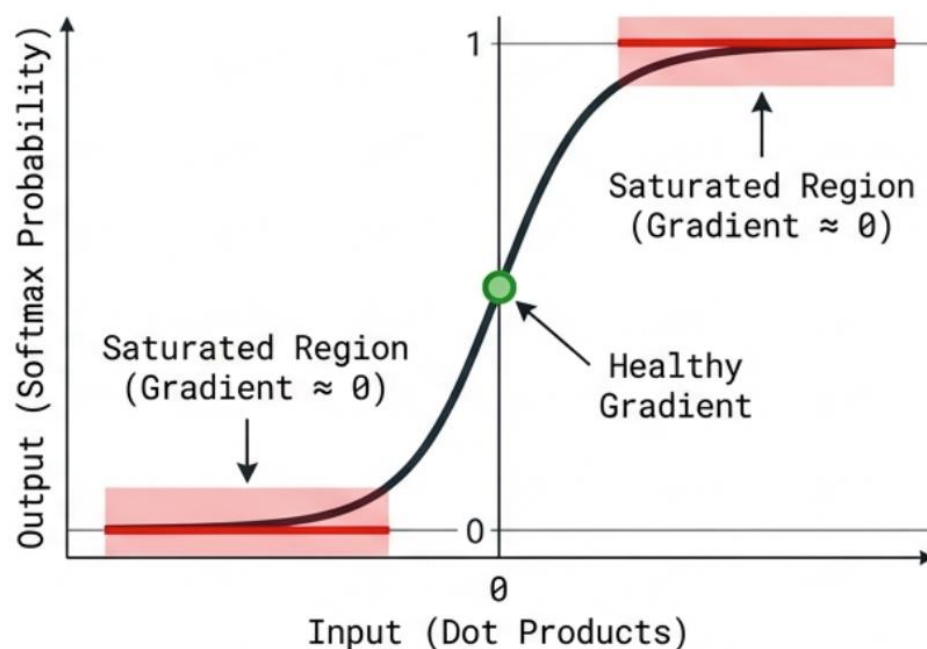
for trainable weight matrices $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$.



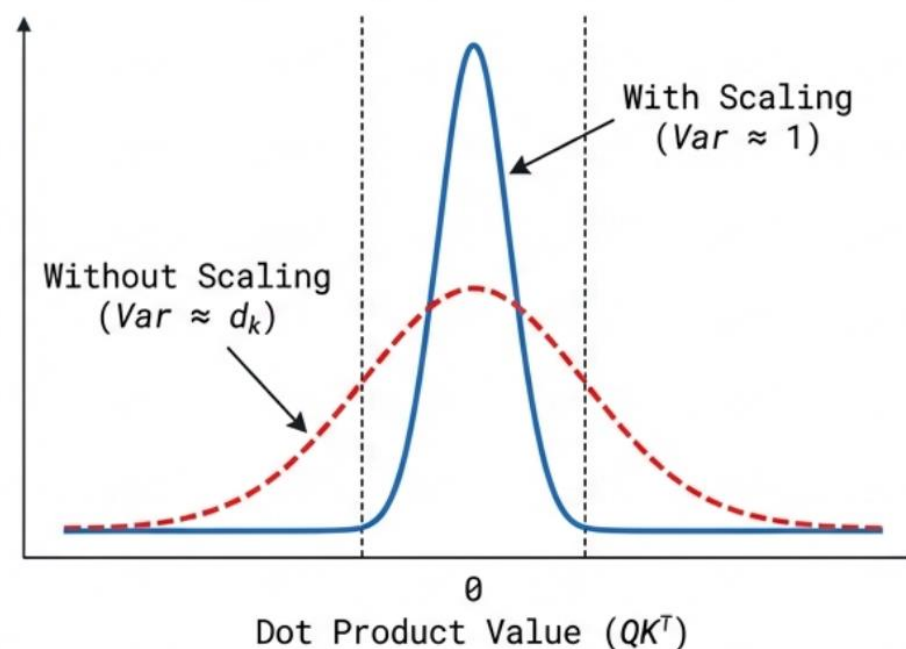
3. Self-Attention: Scaled Dot-Product

Design Choice: Why Scale by $\sqrt{d_k}$?

The Problem: Vanishing Gradients

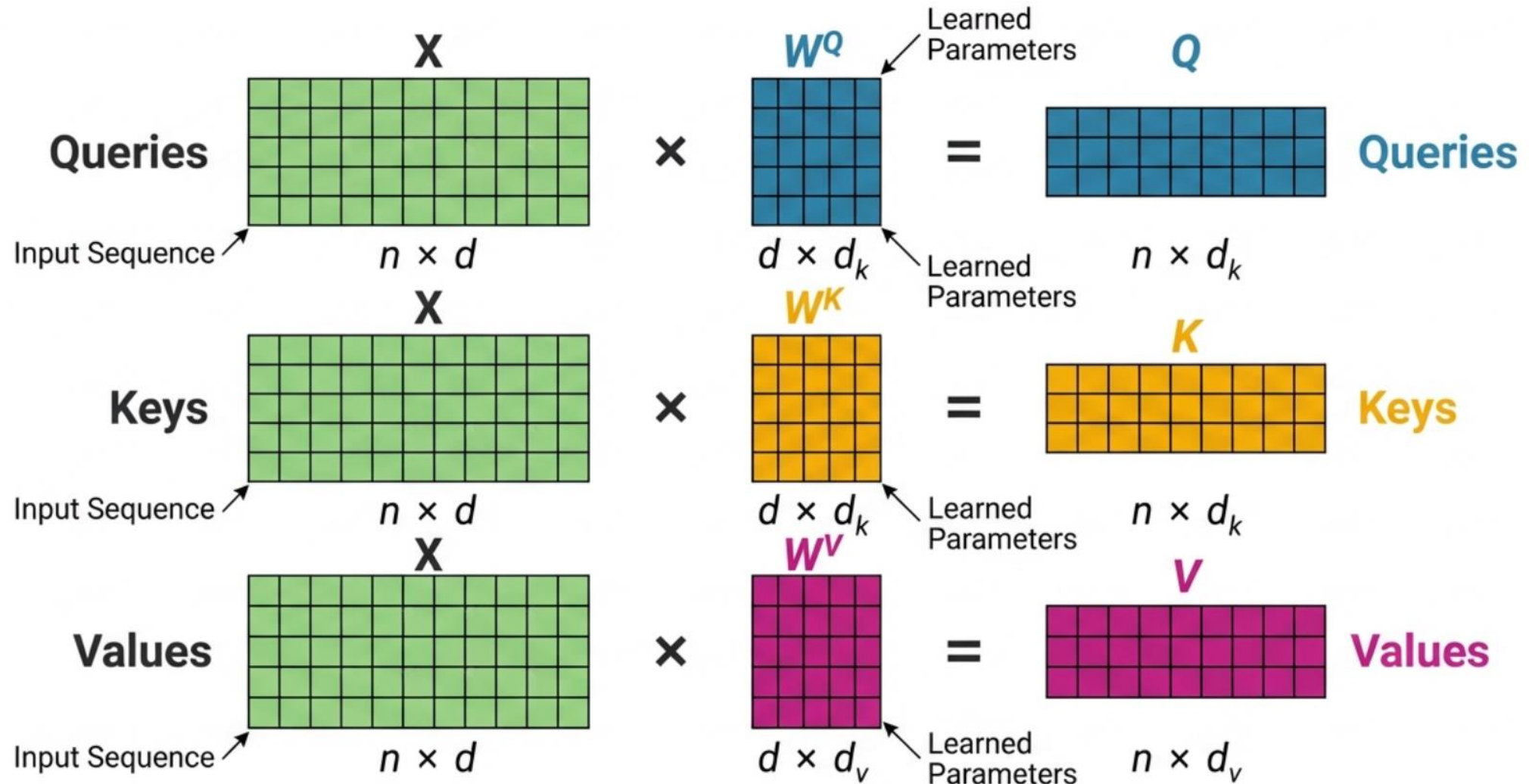


The Solution: Variance Normalization



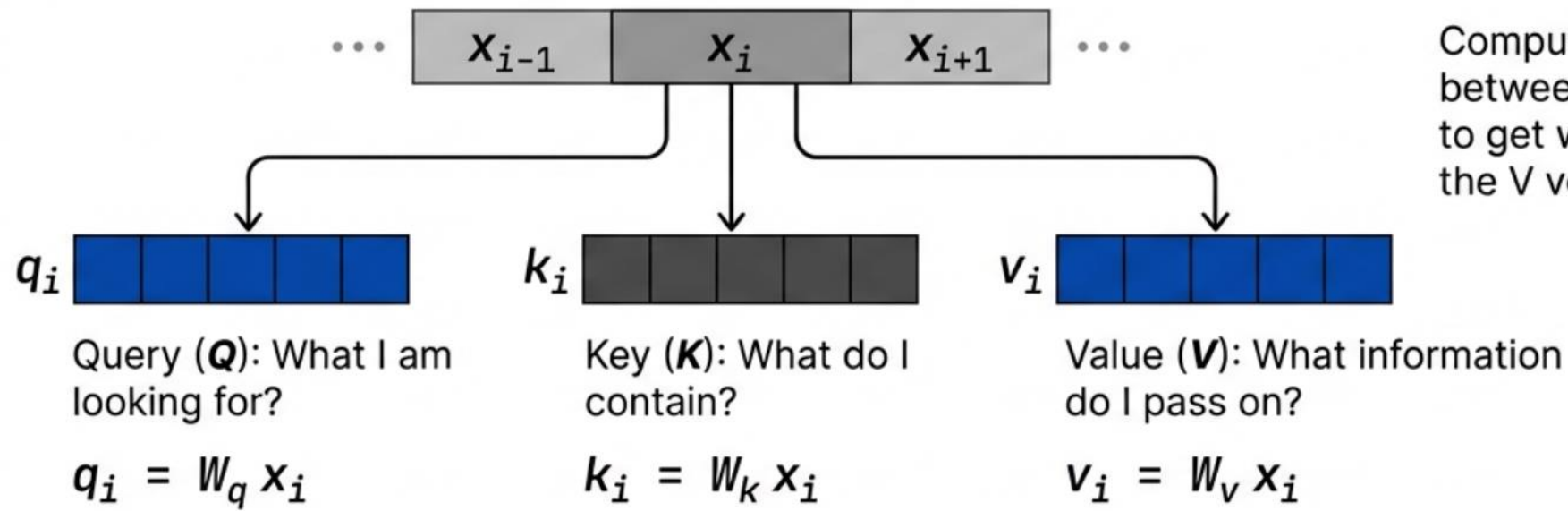
Scaling prevents the dot products from growing too large, ensuring the Softmax stays in the learnable range.

3. Self-Attention



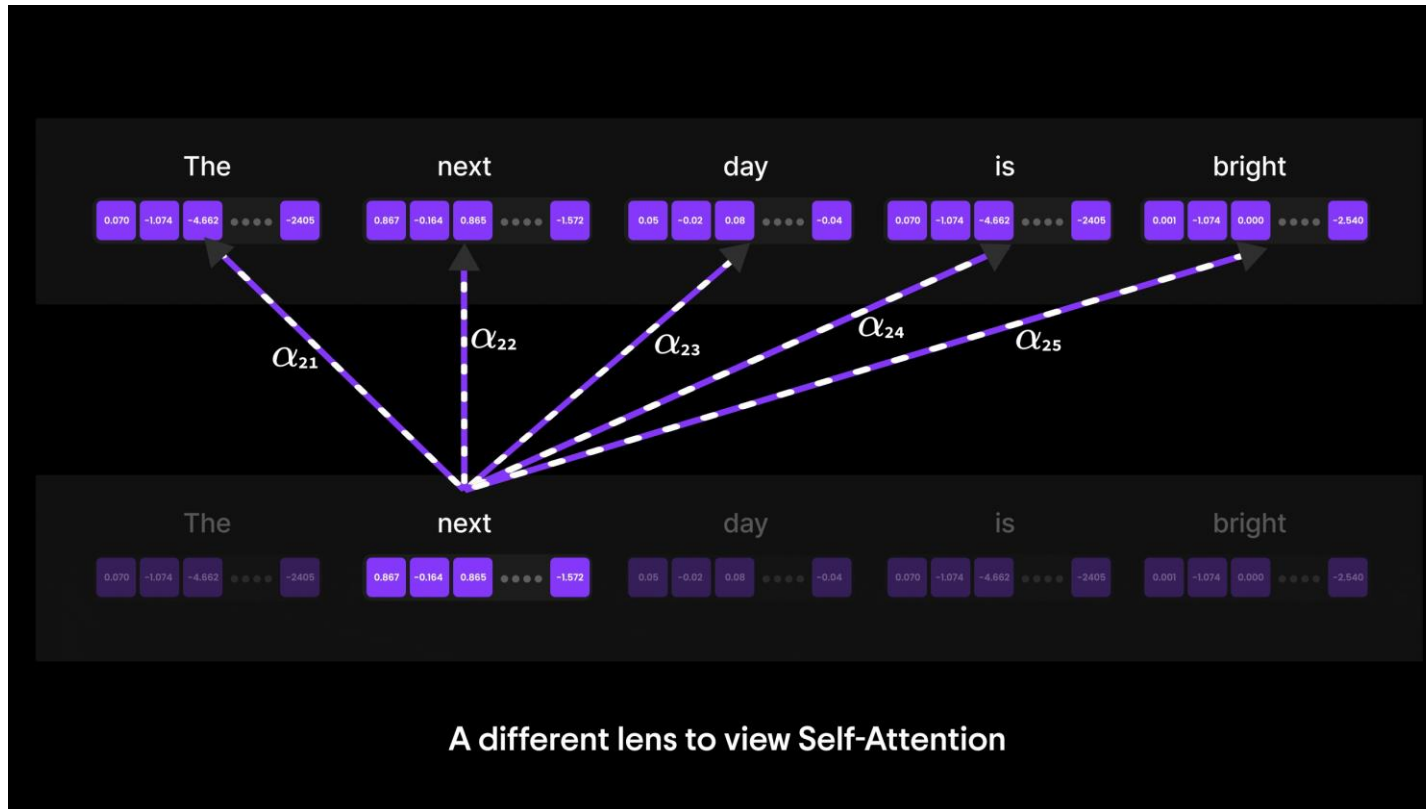
3. Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



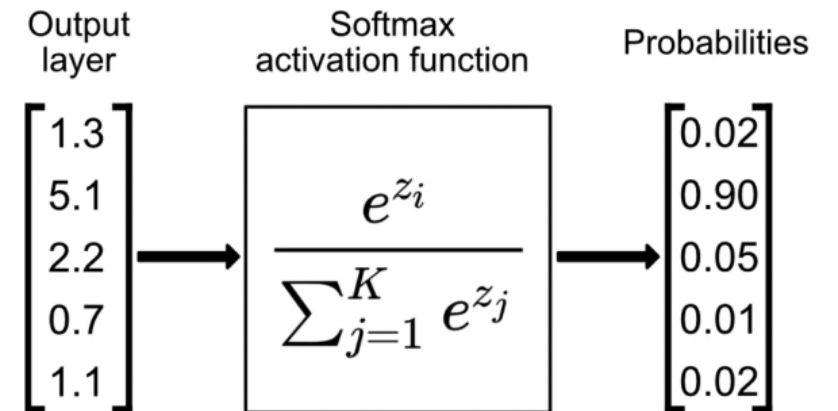
Compute compatibility between Q and K, normalize to get weights, then sum the V vectors.

3. Self-Attention



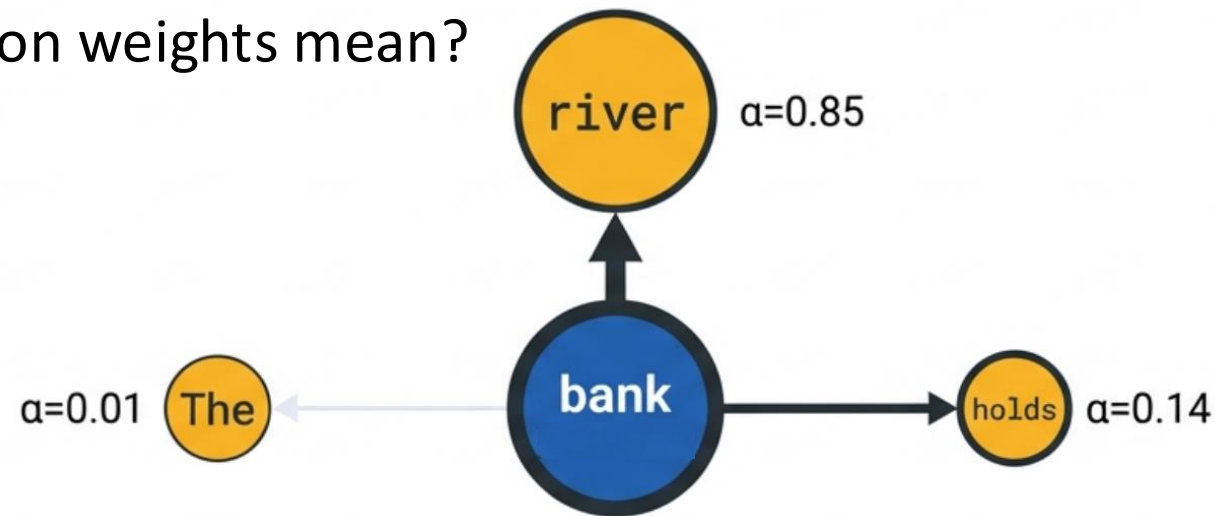
$$\text{softmax}\left(\frac{\mathbf{Q} \times \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

$$= \mathbf{Z}$$



3. Self-Attention

What does attention weights mean?



Weighted Sum

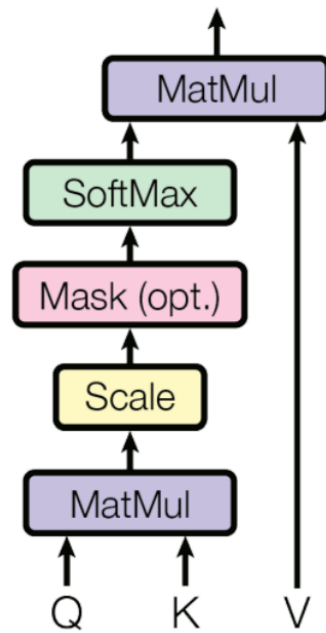
$$\begin{bmatrix} \text{Vector for} \\ \text{"bank"} \\ \text{(new)} \end{bmatrix} = 0.01 \times \text{'The'} + 0.85 \times \text{'river'} + 0.14 \times \text{'holds'}$$

The model learns to “route” content from most relevant context (V) into the current position

3. Multi-head Attention

Multihead attention is used in practice. Instead of using one huge attention, we split queries, keys and values to several groups (similar to how ResNeXt works), compute the attention in each of the groups separately, concatenate the results and multiply them by a matrix W^O .

Scaled Dot-Product Attention



Multi-Head Attention

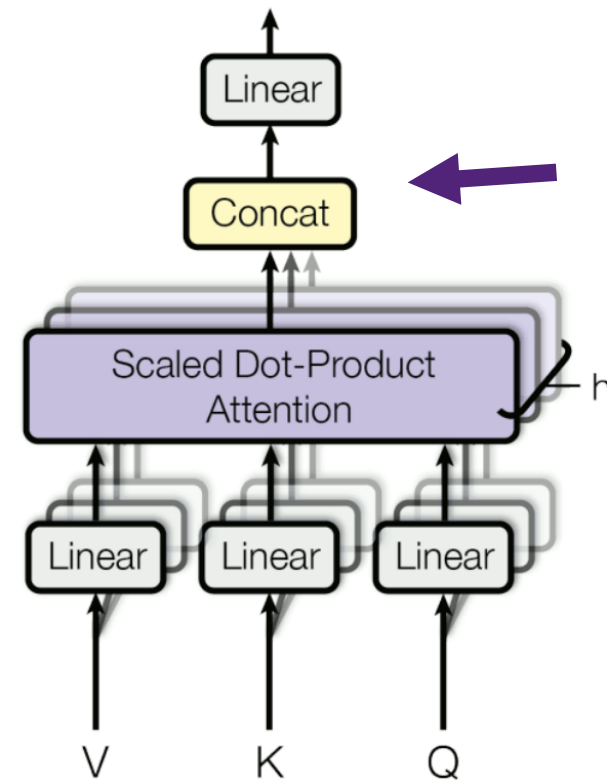
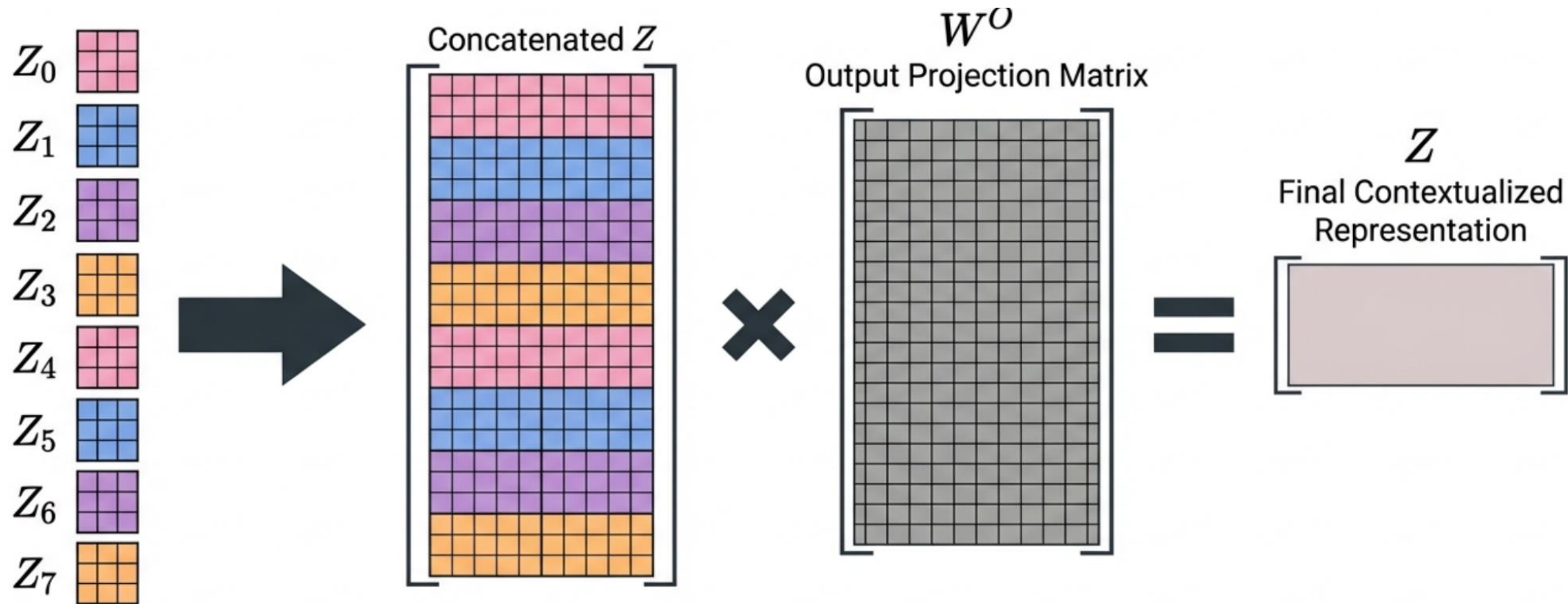


Figure 2 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

3. Multi-head Attention

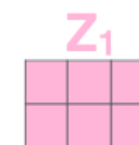
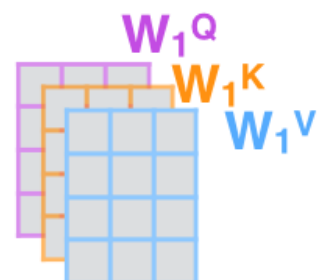
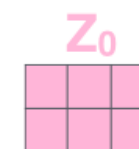
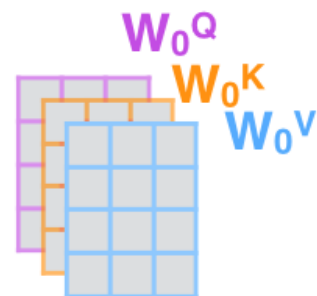


$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Here, we encourage “inter-head” communications across tokens

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

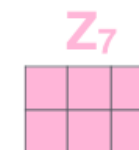
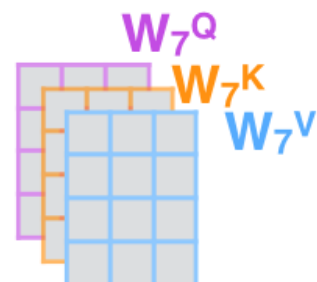
Thinking
Machines



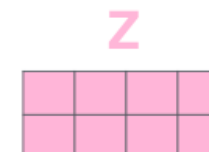
...

...

...

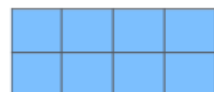


W^O



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R

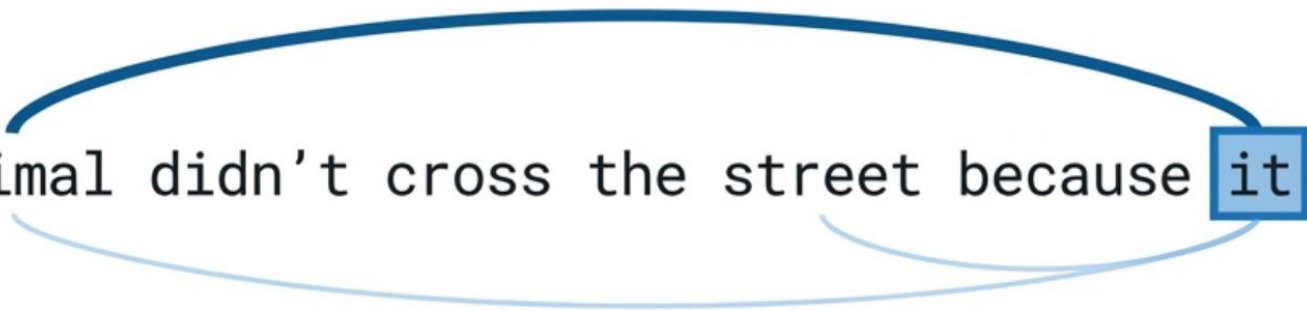


$$M_k = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix}$$

3. Multi-head Attention

What do multiple heads learn?

The animal didn't cross the street because **it** was too tired.



Coreference Resolution

The law was signed.

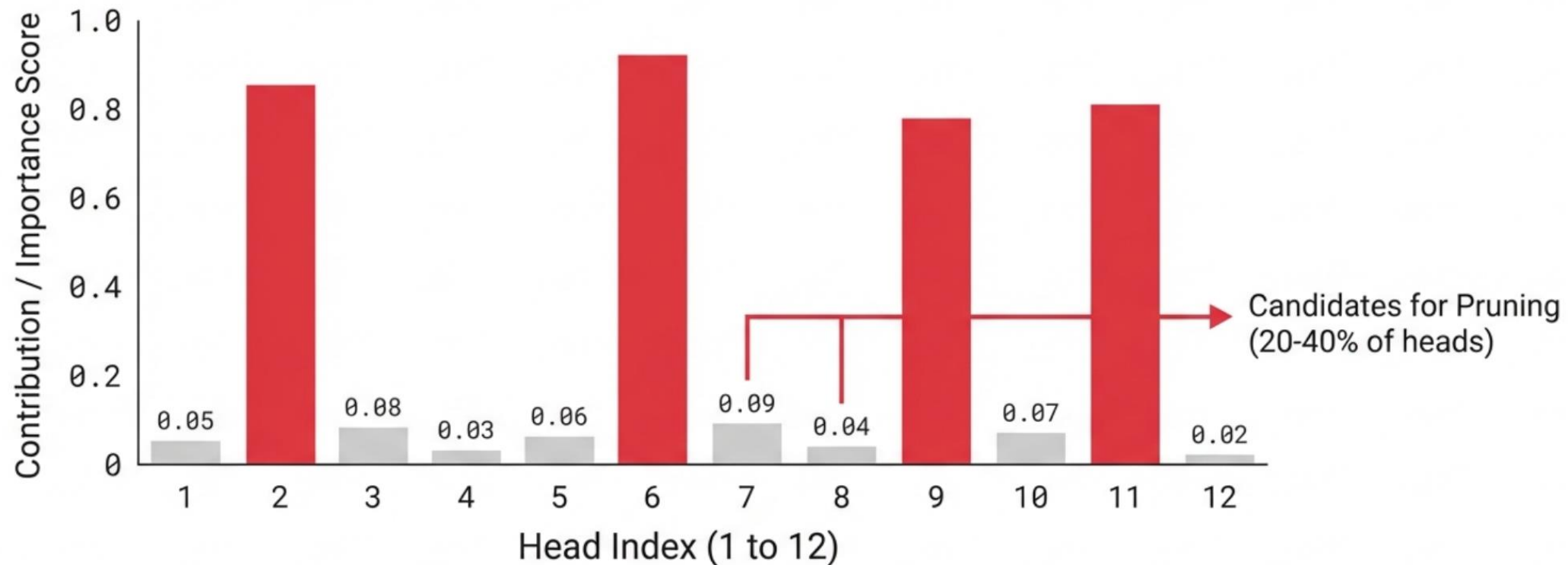


Syntactic Dependency

3. Multi-head Attention

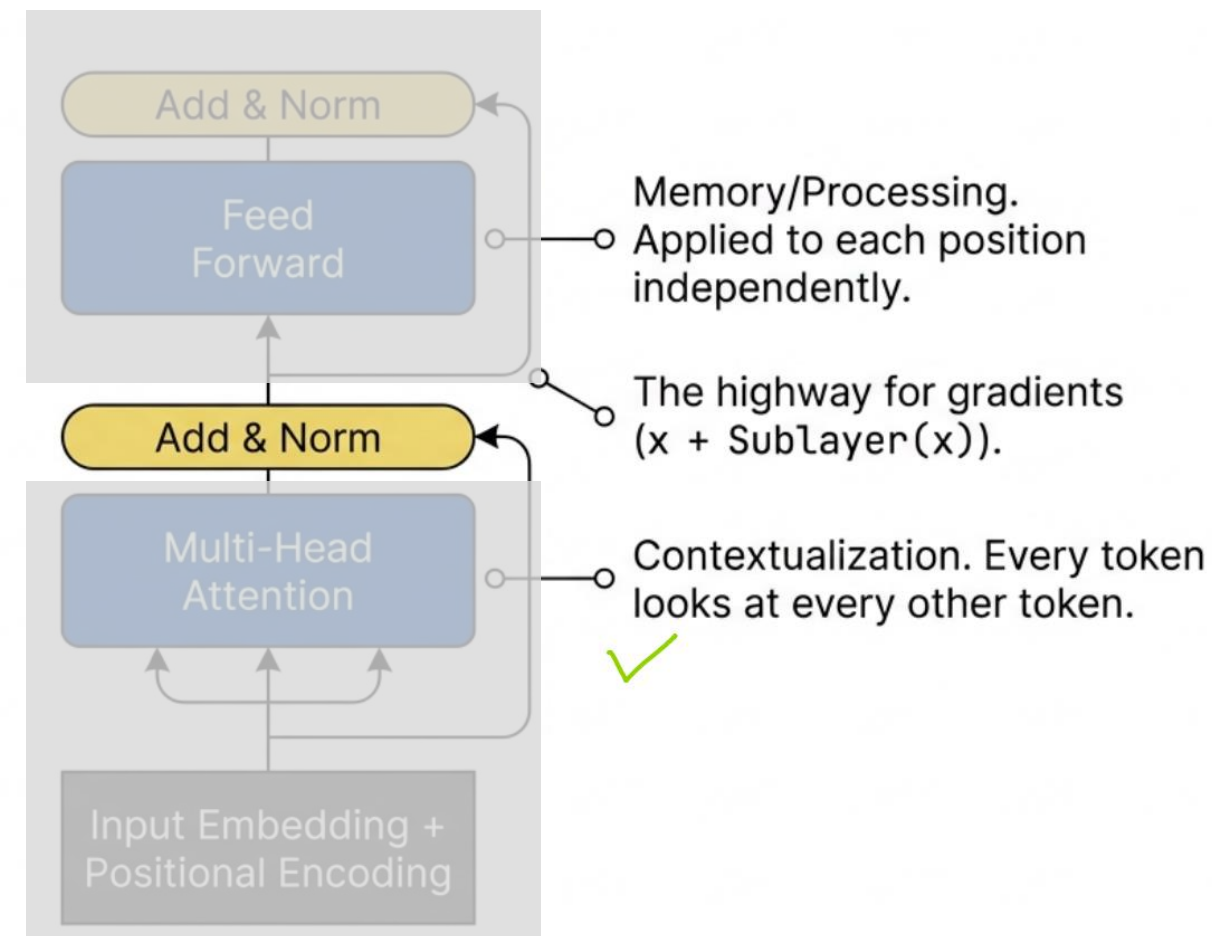
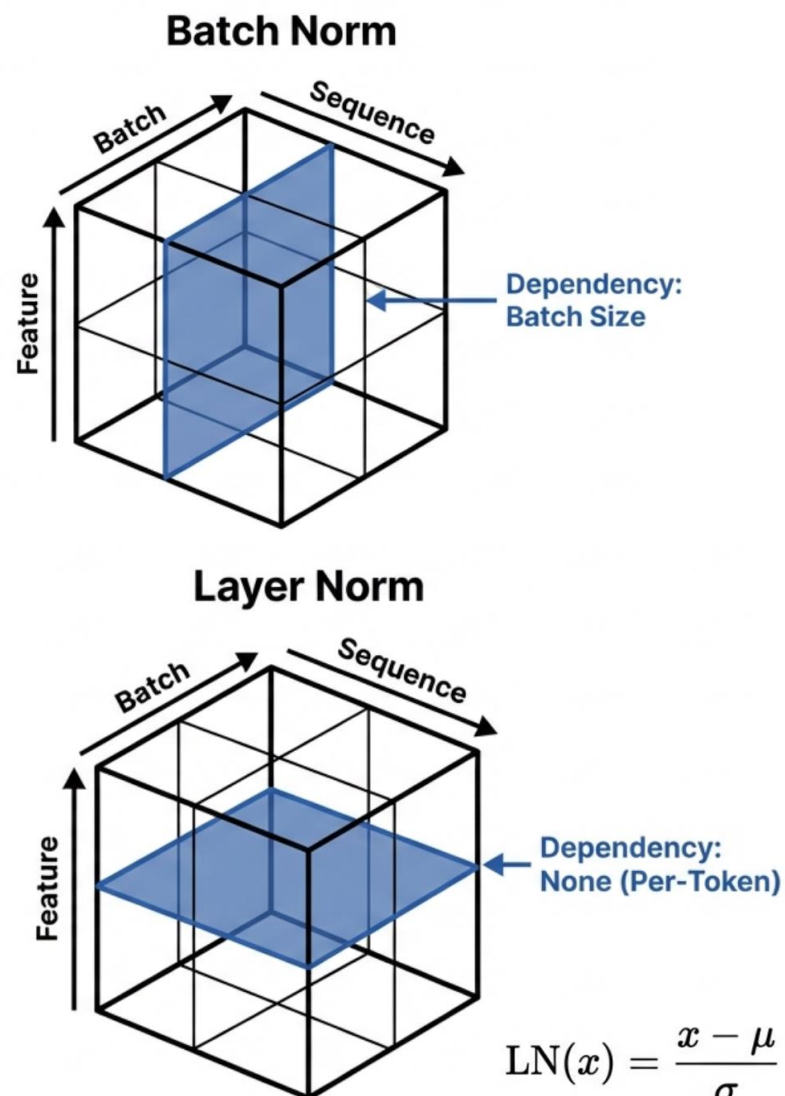
Efficiency Issue: Redundancy & Pruning

Head Importance Distribution



Models are over-parameterized. Many heads learn duplicate or useless patterns -> “dead heads”

3. Self-Attention



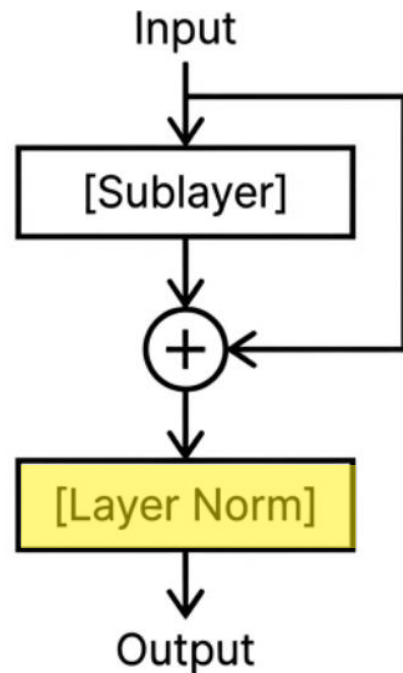
Helps stabilise the gradient for training

3. Self-Attention

Add & Norm

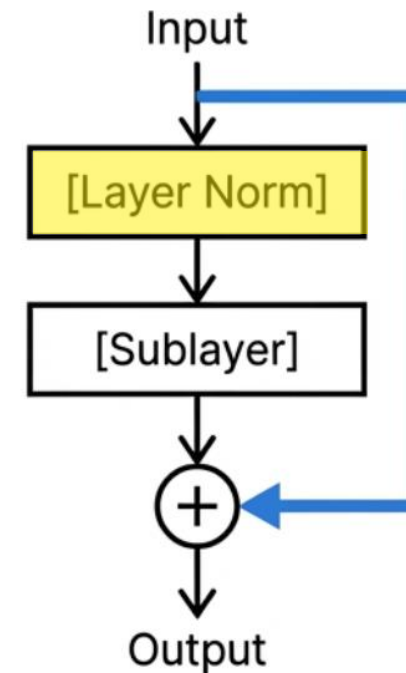
“Add” first or “Norm” first?

Post-Norm (Original)



Hard to train. Gradients can explode.
Requires warm-up.

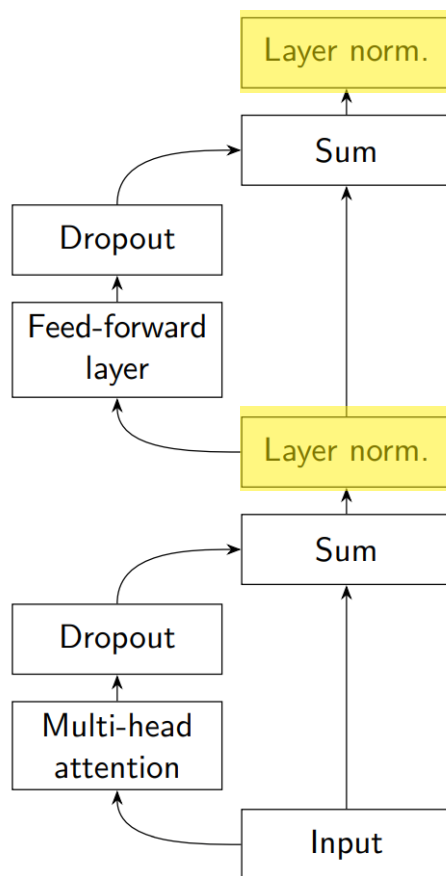
Pre-Norm (Modern)



Stable at scale. Gradients flow directly.
Standard in GPT-3/LLaMA.

3. Self-Attention

Original: Post-normalization

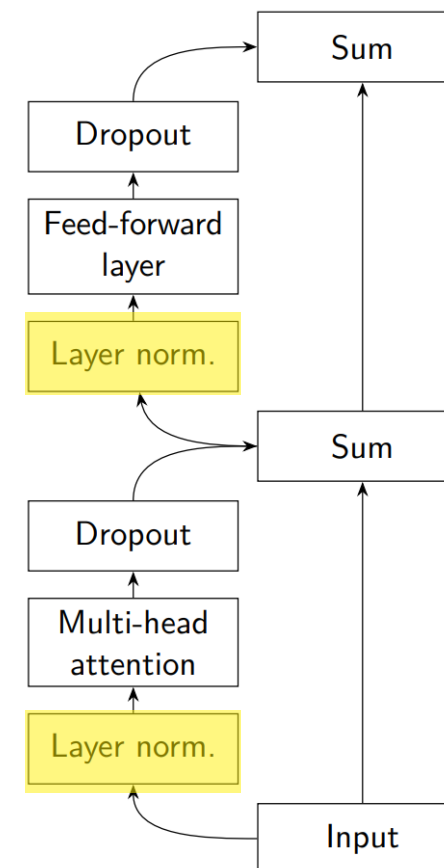


Hard to train. Gradients can explode.
Requires warm-up.

Add & Norm

“Add” first or “Norm” first?

Now more common: Pre-normalization
(Xiong et al., 2020)

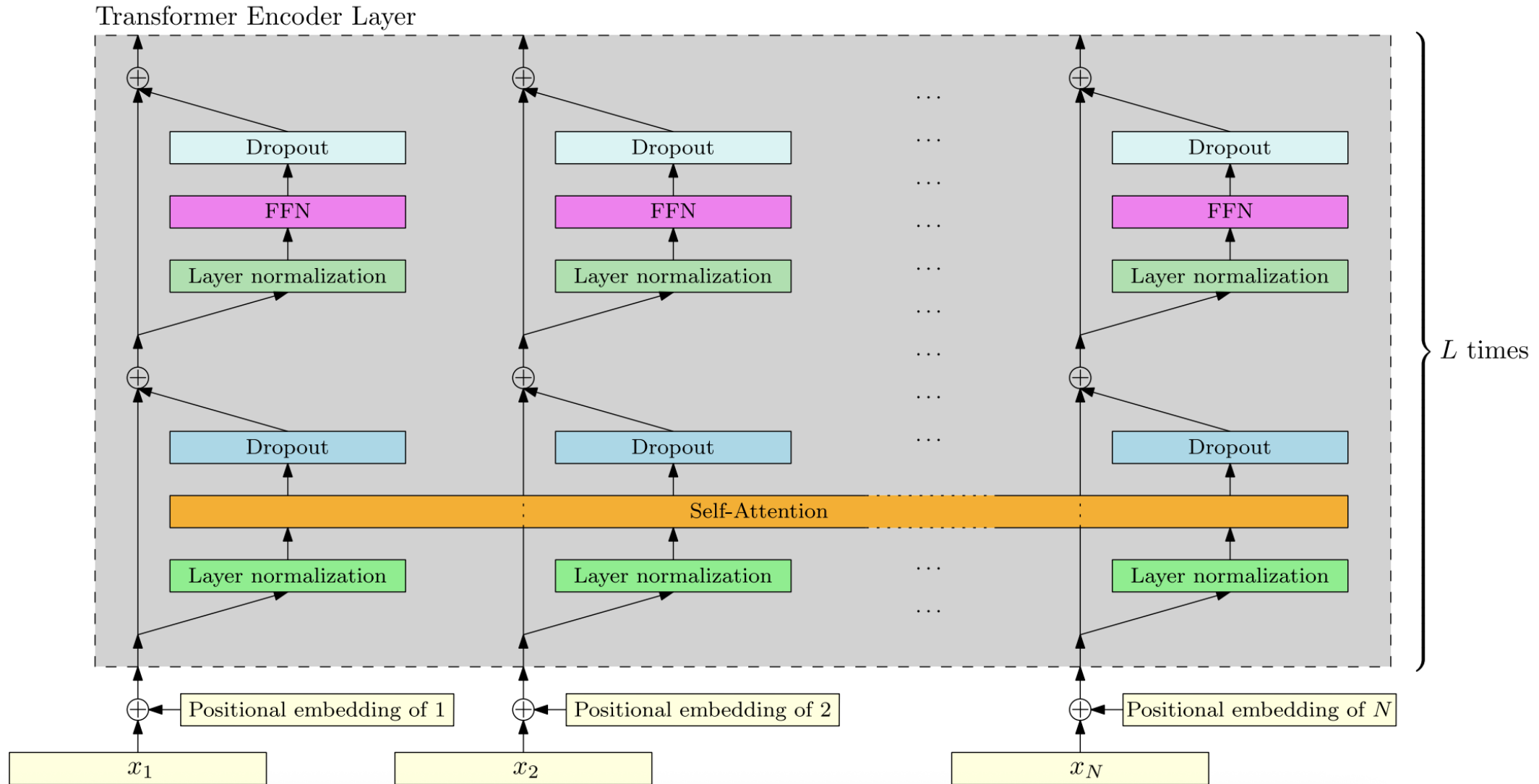


Stable at scale. Gradients flow directly.
Standard in GPT-3/LLaMA.

3. Self-Attention

Add & Norm

“Add” first or “Norm” first?

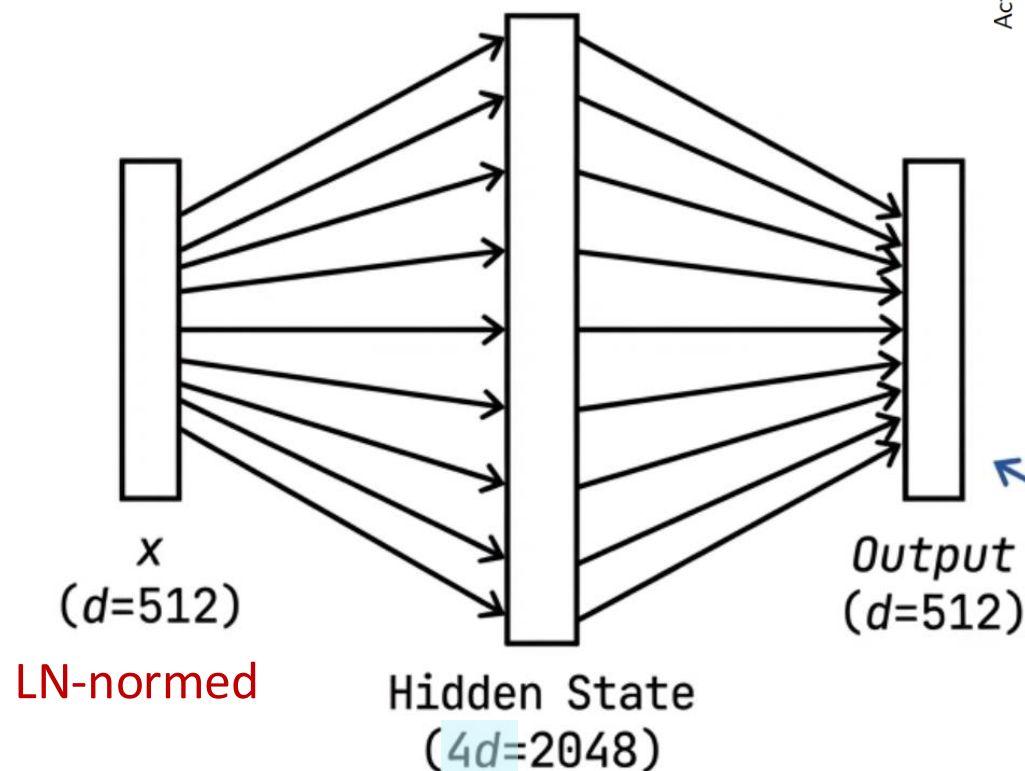


3. Self-Attention

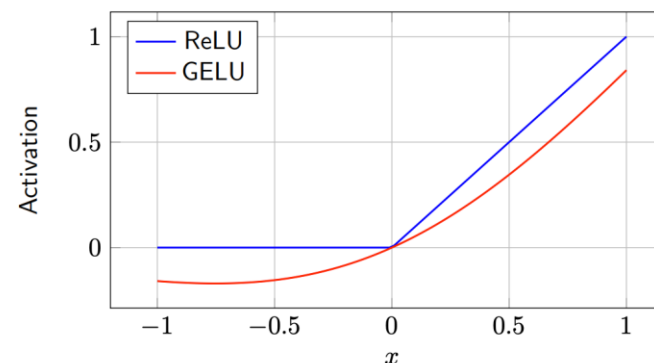
Add & Norm

The Feed-forward Network

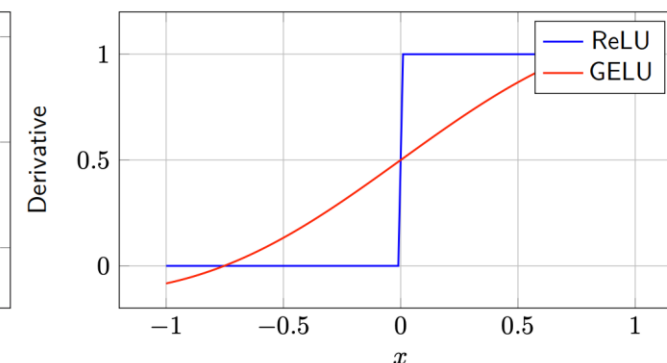
Expansion and Compression



ReLU vs. GELU Activation Functions



Derivatives of ReLU and GELU Activation Functions



$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Non-linear activation (e.g., ReLU)

Applied to each token independently. No mixing between positions.

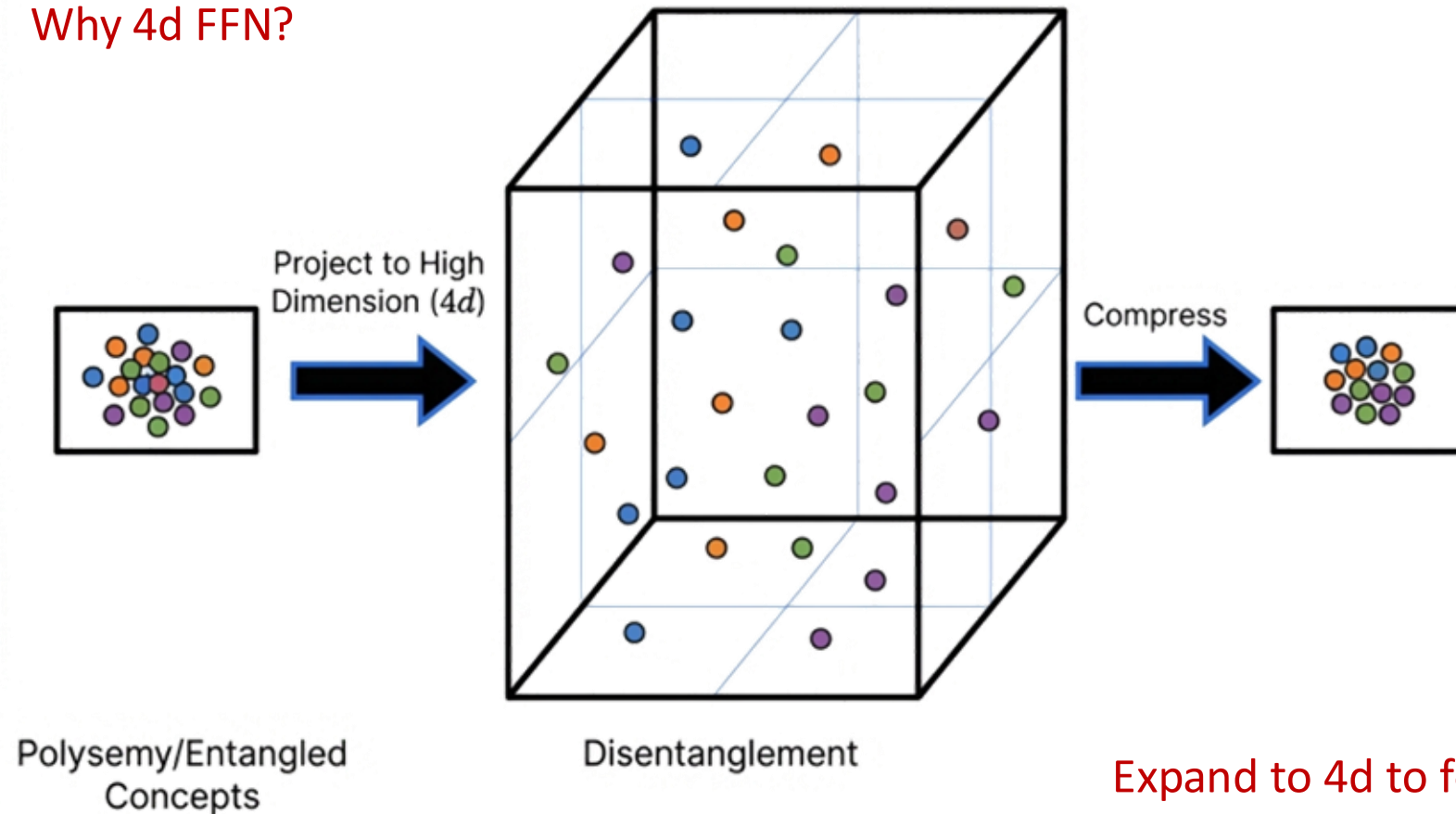
Why 4d?

3. Self-Attention

Add & Norm

The Feed-forward Network

Why 4d FFN?



The Key-Value Memory Hypothesis: The FFN acts as static memory.

- Expansion allows the model to “unfold” overlapping concepts into distinct regions before compression.
- **Cost:** This layer contains **~65% of total model parameters**.

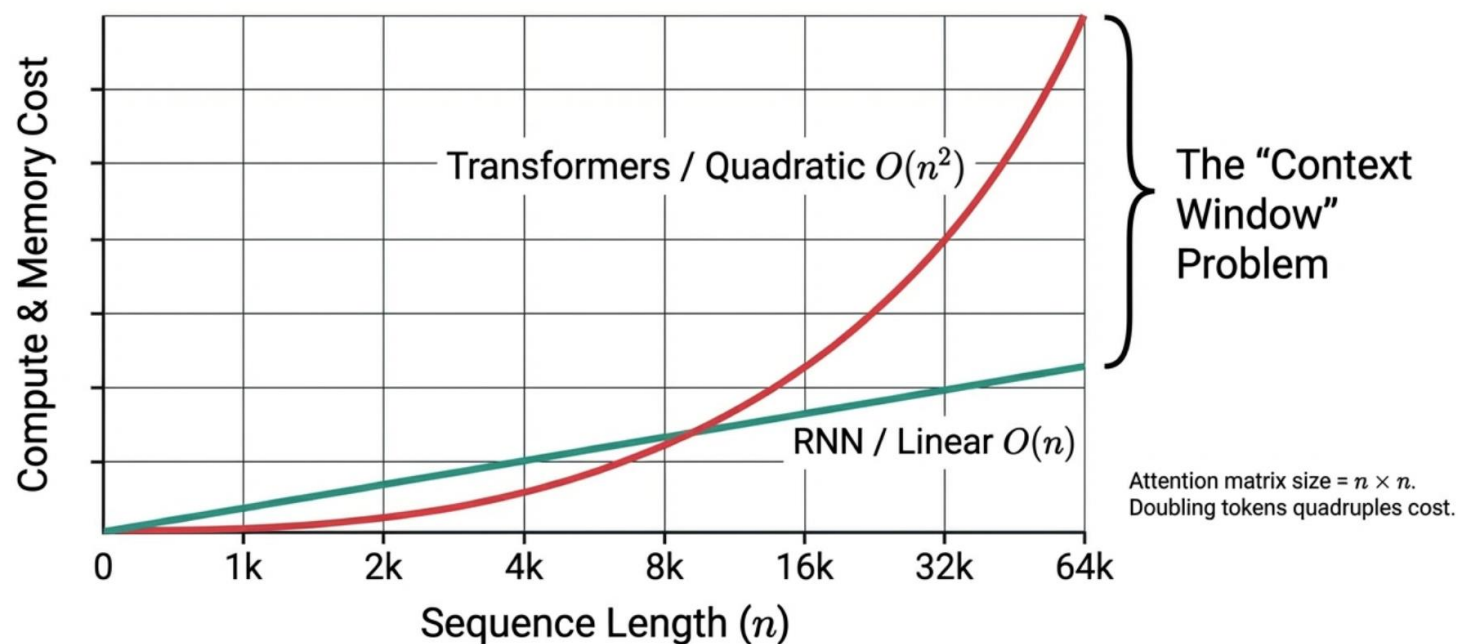
Expand to 4d to form a richer nonlinear basis

-> But then, why not higher d ?

3. Self-Attention

Add & Norm

The Feed-forward Network



- **FFN** dominates the parameter count (Static Memory).
- **Attention** dominates runtime latency as sequence length (N) grows.



$$\begin{aligned}
 &W_q + W_k + W_v + W_o \\
 &= d^2 + d^2 + d^2 + d^2 \\
 &= 4d^2
 \end{aligned}$$

$$\begin{aligned}
 &W_1 + W_2 \\
 &= d * 4d + 4d * d \\
 &= 4d^2 + 4d^2 = 8d^2
 \end{aligned}$$

Component	Parameter Count	Complexity per Token
Attention	$\approx 4d^2$	$O(N^2 \cdot d)$ (Quadratic)
FFN / MLP	$\approx 8d^2$	$O(N \cdot d^2)$ (Linear)

What is a Transformer???

If we zoom in...

1. Tokenization
2. Positional Encoding
3. Self-Attention
4. Output Projection

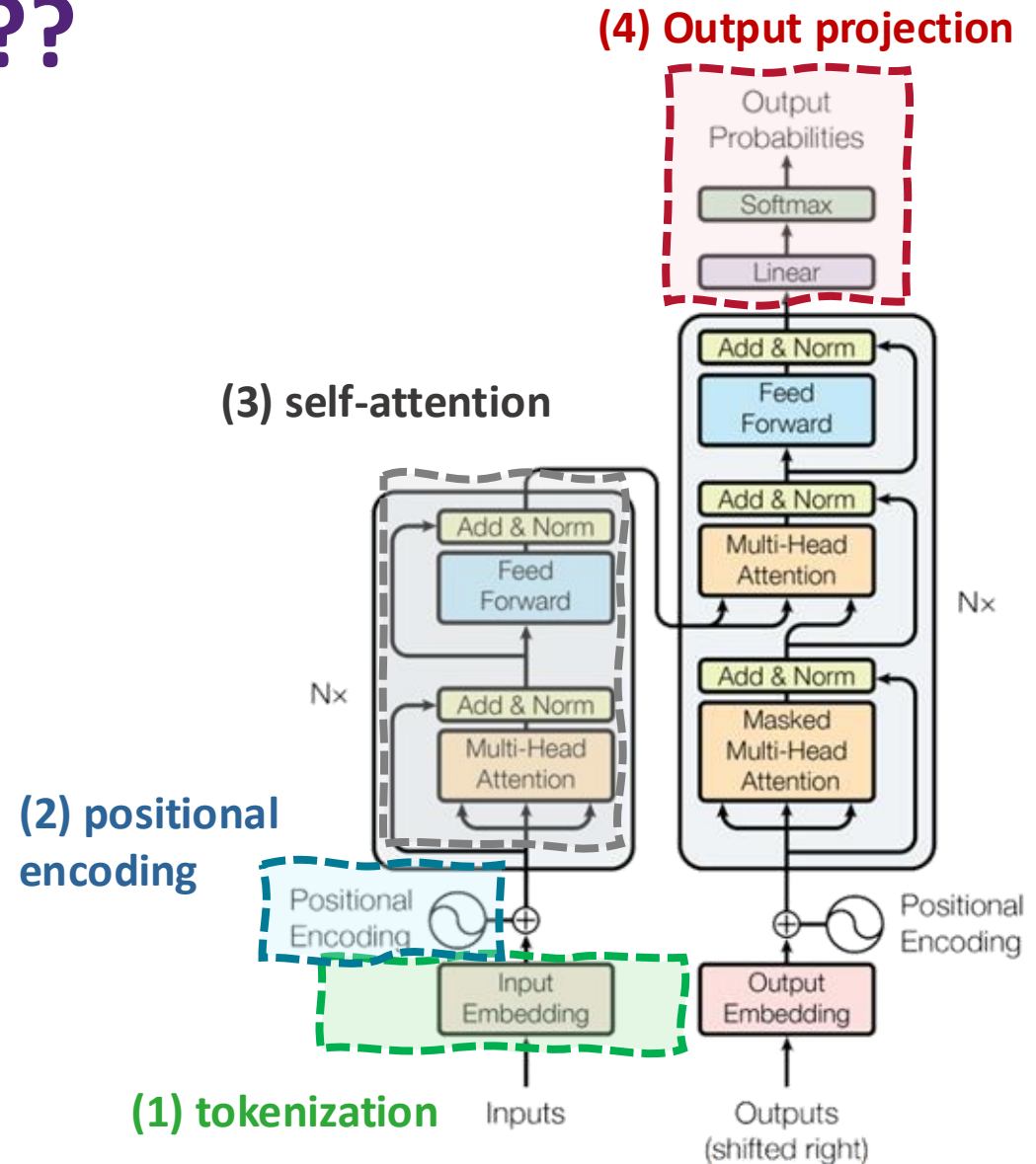


Figure 1 of "Attention Is All You Need", <https://arxiv.org/abs/1706.03762>

Recap

After today's class, you should be able to...

- Describe why the Transformer became a **universal encoder** across text, vision, and other modalities
- Explain the **building blocks** of the Transformer architecture to a non-technical person;
- Describe how **positional encoding** tells the model where each token is; and how **self-attention** works intuitively and mathematically, including queries, keys, values, and scaled dot-product attention

See you at Tutorial Session tomorrow (Llama Vocab, KV cache...)



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

THANKS!

Do you have any questions?

uqyluo@uq.edu.au

The University of Queensland

